

(请大家预习)

第6章第2讲

人工神经网络

Artificial Neural Networks

张 燕 明

ymzhang@nlpr.ia.ac.cn

peopleucas.ac.cn/~ymzhang

模式分析与学习课题组(PAL)

多模态人工智能系统实验室 中科院自动化所

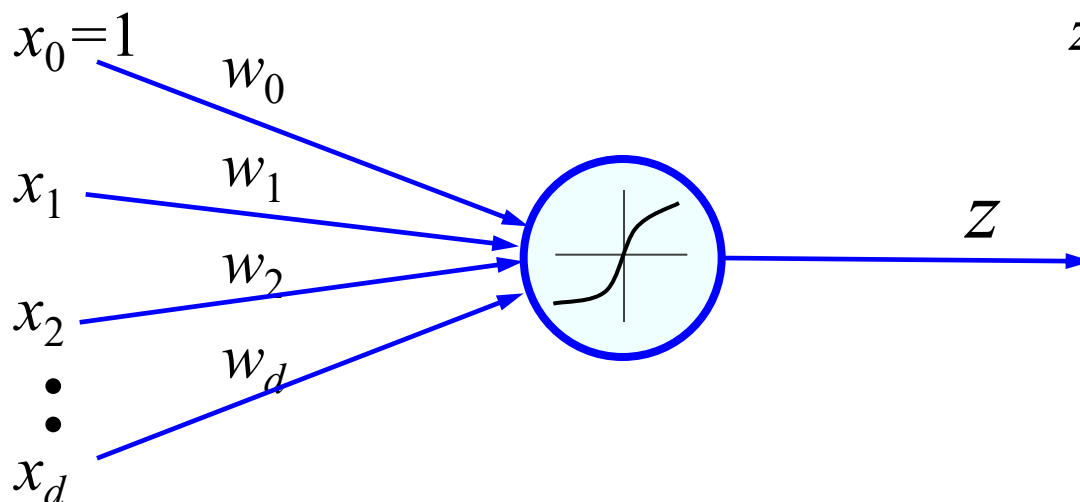
助教： 杨 奇 (yangqi2021@ia.ac.cn)

张 涛 (zhangtao2021@ia.ac.cn)

第7讲内容回顾

- 神经元模型

$\mathbf{x}$: 输入向量
 $\mathbf{w}$: 权重向量
 net : 净输入
 z : 输出



加权求和: $net = \sum_{i=0}^d w_i x_i$

非线性变换: $z = f(net)$ $f(\cdot)$ 称为激活函数

神经元模型

- 激活函数 $f(\bullet)$ 的性质

2~4为非必要条件

1. 非线性：使神经网络具有非线性

2. 饱和性： $\lim_{s \rightarrow \infty} f'(s) = 0$

- ✓ 将无限域变换到有限范围

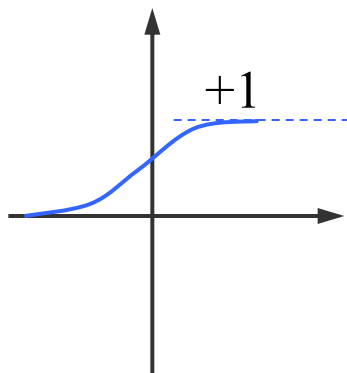
- ✓ 但会使梯度消失，从而影响网络的训练

3. 连续、可导

4. 单调性

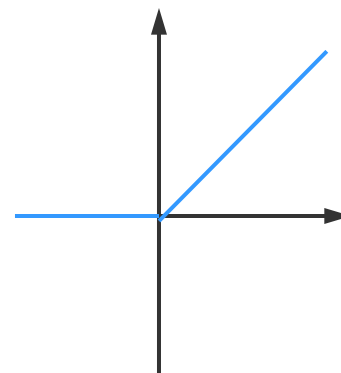
Sigmoid

$$f(s) = \frac{1}{1 + e^{-s}}$$

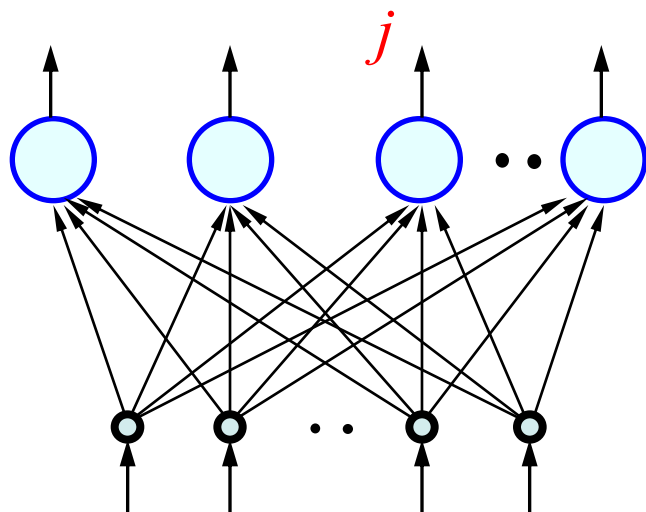


ReLU

$$f(s) = \begin{cases} s & s \geq 0 \\ 0 & s < 0 \end{cases}$$



感知器模型

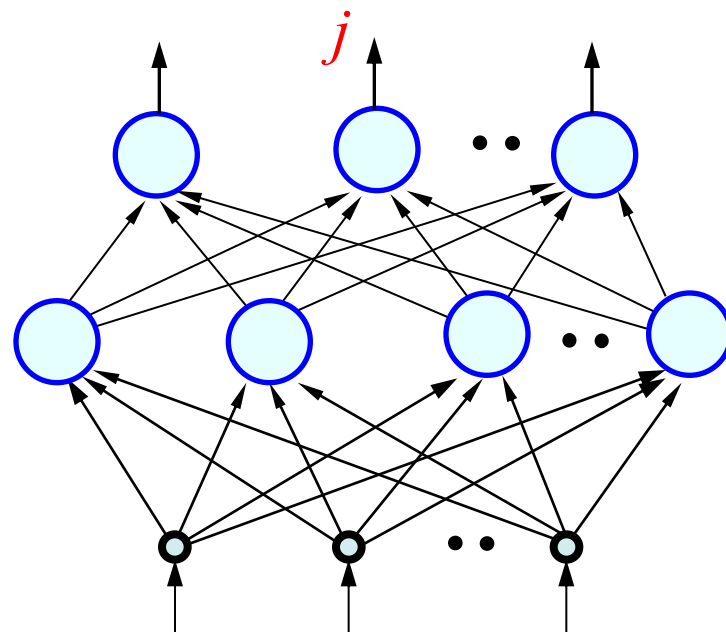


单层感知器

Single Layer Perceptron

$$z_j = f\left(\sum_{i=0}^d w_{ij} x_i\right)$$

- ✓ 由输入层($d+1$ 个节点)、输出层(c 个节点)、 $0 \sim L$ 个隐含层组成
- ✓ 信息从底层向高层传递（前馈）
- ✓ 相邻层之间全连接、没有层内和跨层连接



多层感知器

Multi-Layer Perceptron, MLP

$$z_j = f\left(\sum_h w_{hj} f\left(\sum_{i=0}^d w_{ih} x_i\right)\right)$$

感知器模型的学习

- 反向传播算法 (Back-Propagation, BP)
 - 任务：根据训练样本对 $\{(\mathbf{x}^k, \mathbf{t}^k)\}$ ，确定MLP的权重 $\{w\}$
 - 方法：用梯度下降法优化误差函数 $E(\{w\})$

$$w = w - \eta \frac{\partial E}{\partial w}$$

设 j, h 分别为输出层和最后一个隐层中的结点：

对连接 $h \rightarrow j$

$$\frac{\partial E}{\partial w_{h \rightarrow j}} = \frac{\partial E}{\partial net_j} \frac{\partial net_j}{\partial w_{h \rightarrow j}} = \boxed{\frac{\partial E}{\partial net_j}} y_h$$

net_j : j 结点的净输入
 y_h : h 结点的输出

$$\boxed{\frac{\partial E}{\partial net_j}} = (z_j - t_j) f'(net_j)$$

假设 $E(w) = \frac{1}{2} \sum_k \|\mathbf{z}^k - \mathbf{t}^k\|^2$

感知器模型的学习

- 反向传播算法 (Back-Propagation, BP)
 - 任务: 根据训练样本对 $\{(\mathbf{x}^k, \mathbf{t}^k)\}$, 确定MLP的权重 $\{w\}$
 - 方法: 用梯度下降法优化误差函数 $E(\{w\})$

设 h, h', h'' 分别为第 $l, l+1, l+2$ 层中的结点:

对连接 $h \rightarrow h'$
$$\frac{\partial E}{\partial w_{h \rightarrow h'}} = \frac{\partial E}{\partial net_{h'}} \frac{\partial net_{h'}}{\partial w_{h \rightarrow h'}} = \boxed{\frac{\partial E}{\partial net_{h'}}} y_h$$

net_h : h 结点的净输入
 y_h : h 结点的输出

对结点 h'
$$\boxed{\frac{\partial E}{\partial net_{h'}}} = \sum_{h''} \frac{\partial E}{\partial net_{h''}} \frac{\partial net_{h''}}{\partial y_{h'}} \frac{\partial y_{h'}}{\partial net_{h'}} = \boxed{f'(net_{h'})} \sum_{h''} \boxed{\frac{\partial E}{\partial net_{h''}}} w_{h' \rightarrow h''}$$

通过从后向前的反向过程计算 $\frac{\partial E}{\partial net_h}$

感知器模型的学习

- 反向传播算法 (Back-Propagation, BP)

Stochastic Backpropagation

- 1 begin initialize: n_H , $\mathbf{w}$, η , criterion θ , $k=0$
- 2 do $k \leftarrow k + 1 \pmod{n}$
- 3 $\mathbf{x}^k$, randomly chosen a sample
- 4 $w_{hj} \leftarrow w_{hj} - \eta \frac{\partial E}{\partial net_j^k} y_h^k$ $w_{ih} \leftarrow w_{ih} - \eta \frac{\partial E}{\partial net_h^k} x_i^k$
- 5 until $\| \nabla E(\mathbf{w}) \| < \theta$
- 6 return $\mathbf{w}$
- 7 end

第五节 BP算法讨论以 及神经网络训练

6.5.1 准则函数

- 回归问题

- $\mathbf{x}^k$ 的目标值 t^k 为实数标量，对应网络的输出层只需一个结点
 - 如：温度、血压、价格、距离

- 多分类问题

- 若类别数为 c ，则输出层的结点个数为 c ，对应 c 个判别函数，按最大判别函数准则分类。
- 对于训练样本 $\mathbf{x}^k$ ，如果它属于第 i 类，则其目标值 $\mathbf{t}^k$ 可以定义为一个 c 维向量，该向量只有第 i 个元素为 1，其余元素的值均为 0 (one-hot encoding)。
- 这 c 个值分配给 c 个输出结点，作为训练目标。

6.5.1 准则函数

z_j^k 为预测 $\mathbf{x}^k$ 属于第 k 类的后验概率，通常由 softmax 层输出

$$z_j = \frac{\exp(y_j)}{\sum_{i=1}^c \exp(y_i)}$$

- 常用的准则(损失、误差)函数

平方误差准则：

$$E_{MSE}(\mathbf{w}) = \sum_{k,j} \left(t_j^k - z_j^k \right)^2$$

交叉熵准则：

$$E_{CE}(\mathbf{w}) = - \sum_{k,j} t_j^k \ln(z_j^k)$$

Minkowski 误差准则：

$$E_{Mink}(\mathbf{w}) = \sum_{k,j} \left| t_j^k - z_j^k \right|^R, \quad 1 \leq R < 2$$

6.5.2 激活函数

- 激活函数

- 激活函数应是非线性的。
- 激活函数是连续可导的。
- 激活函数最好是单调的。否则，误差函数会包含更多的局部极小值点，从而增加训练难度。
- Sigmoid函数（以及双曲正切函数）满足上述性质、但两侧饱和性质容易造成梯度消失。
- 现在用得较多的是ReLU函数及其变种。

6.5.3 隐含层数

- Universal Approximation Theorem (Cybenko 1989)
 - 对于一个定义在有限维实向量空间上的任意连续函数 $f: R^n \rightarrow R$ ，以及任意精度 $\epsilon > 0$ ，只要选取一个非线性激活函数(如sigmoid, tanh, ReLU)，就存在一个单隐层前馈神经网络

$$g(\mathbf{x}) = \sum_{i=1}^m \alpha_i \sigma(w_i^T \mathbf{x} + b_i)$$

使得(对任一紧集 K):

$$\sup_{\mathbf{x} \in K} |g(\mathbf{x}) - f(\mathbf{x})| < \epsilon.$$

- 意义：理论上说明了神经网络的表示能力非常强大
- 局限：仅是存在性定理

6.5.3 隐含层数

- 隐含层数设定
 - 虽然单隐层神经网络只要足够宽，就能逼近任何连续函数；但深层网络通过复用中间结构，具有更高的表示效率。
 - 因此，实践中仍然采用多层网络，网络层数的选取依经验和情况而定。

M. Telgarsky. Benefits of depth in neural networks. COLT, 2016.

6.5.4 结点个数

- 隐含层结点数设定
 - 各层结点数的选择对网络的性能影响较大
 - 隐含层结点数太少，网络难以建立复杂的判别界面；取得太多，判决界面仅包含训练样本点而失去推广能力。
 - 通常隐含层结点的个数设置得较大。待网络训练之后，考察有无需要减少结点数的可能。
 - 压缩神经网络
 - 稀疏连接的神经网络
 - dropout技术

6.5.5 初始权重

- 初始权重
 - 在批处理权重更新算法中，初始权重的更新值 $\Delta w_{ij} \neq 0$ ，否则不会产生学习。（最后一个隐含层至输出层）
 - 权重可以为正，也可以为负。
- 初始化方法
 - 全0、常数初始化 ✕
 - 随机初始化 ✓
 - 从均匀分布、高斯分布中随机选择初始值

$$w_{ij} \sim U(-w_0, w_0)$$

$$w_{ij} \sim N(\mu=0, \sigma=w_0)$$

6.5.5 初始权重

- 初始化方法(接上页)

- 随机初始化 ✓

n : i 结点所在层的结点个数

m : j 结点所在层的结点个数

- For Sigmoid and Tanh (Xavier Initialization)

$$w_{ij} \sim U(-\frac{1}{\sqrt{n}}, \frac{1}{\sqrt{n}}) \text{ 或 } w_{ij} \sim U(-\frac{\sqrt{6}}{\sqrt{n+m}}, \frac{\sqrt{6}}{\sqrt{n+m}})$$

- For ReLU (He Initialization)

$$w_{ij} \sim N(\mu=0, \sigma=\frac{\sqrt{2}}{\sqrt{n}})$$

- 预训练初始化 ✓

Xavier Glorot, Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. AISTATS, 2010.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun. Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification. ICCV, 2015.

6.5.6 正则化技术

- 目标函数正则化

- 防止网络出现 overfitting 的一种有效方法是采用正则化技术

- 权重2范数正则化技术修正函数：

$$E_{new}(\mathbf{w}) = E(\mathbf{w}) + \frac{\varepsilon}{2\eta} \mathbf{w}^T \mathbf{w}$$


$$w_{ij} = w_{ij} - \eta \frac{\partial E_{new}(\mathbf{w})}{\partial w_{ij}} = w_{ij} - \eta \left(\frac{\partial E(\mathbf{w})}{\partial w_{ij}} + \frac{\varepsilon}{\eta} w_{ij} \right) = (1 - \varepsilon) w_{ij} - \eta \frac{\partial E(\mathbf{w})}{\partial w_{ij}}$$

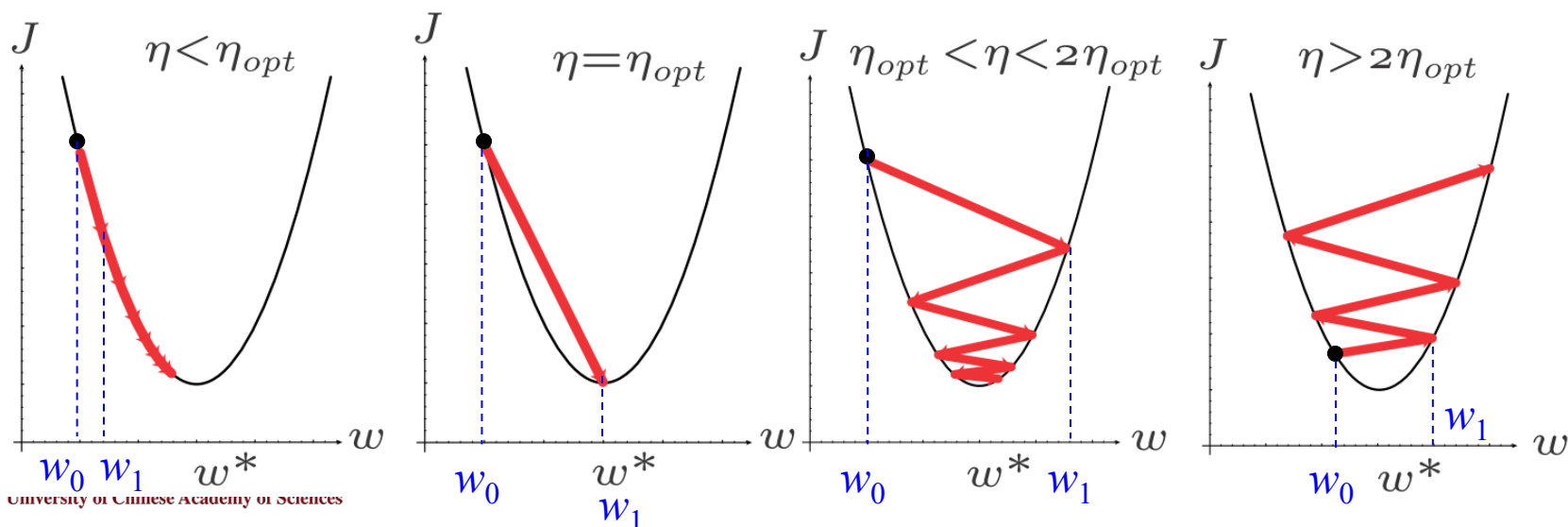
权值衰减

6.5.7 学习率

$$w = w - \eta \frac{\partial E}{\partial w}$$

- 学习率（梯度更新步长）

- 学习率 η 太小，则收敛较慢；过大则不稳定。
- 调节参数的准则是检查某特定权重修正是否确实降低了误差函数的值：
 - 如果不是，则 η 应该减小。
 - 如果连续几次迭代均降低了误差函数，则表明所选的 η 值可能太保守了，应将 η 增加一个量。



6.5.8 梯度下降法的改进

- 附加冲量项(gradient descent with momentum)
 - 在最优点附近，误差表面可能会较平坦，梯度下降法收敛较慢。可以考虑更多的历史迭代信息。
 - 权重更新可以参考以下公式来进行 ($0 \leq \beta < 1$):

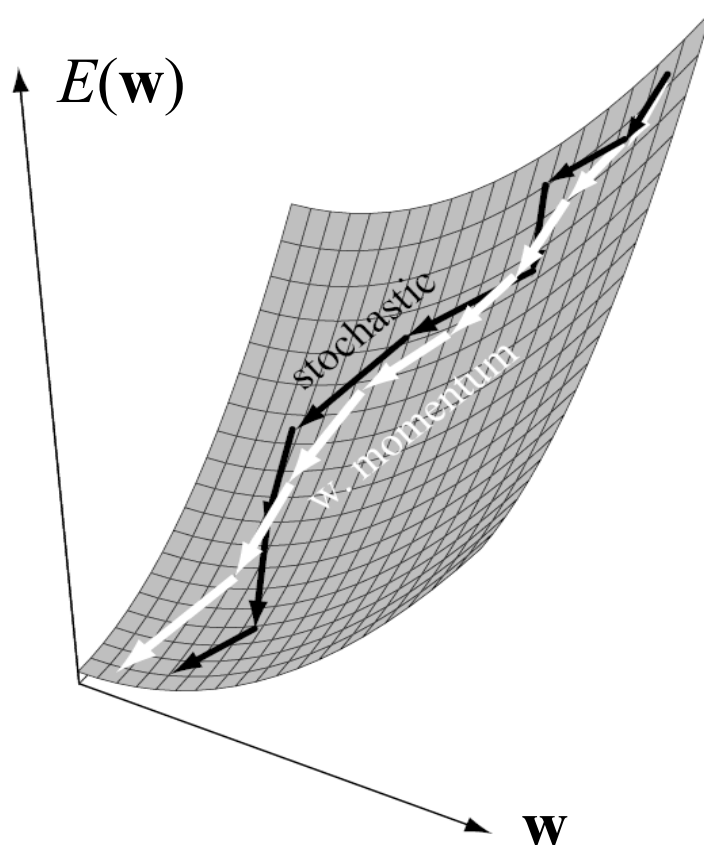
$$\mathbf{m}_t = \beta \mathbf{m}_{t-1} + (1 - \beta) \mathbf{g}_t \quad \text{where} \quad \mathbf{g}_t = \left. \frac{\partial E(\mathbf{w})}{\partial \mathbf{w}} \right|_{\mathbf{w}=\mathbf{w}_t}$$
$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \mathbf{m}_t$$

$$\mathbf{m}_t = \beta \mathbf{m}_{t-1} + (1 - \beta) \mathbf{g}_t \Rightarrow \mathbf{m}_t = (1 - \beta)(\mathbf{g}_t + \beta \mathbf{g}_{t-1} + \beta^2 \mathbf{g}_{t-2} + \beta^3 \mathbf{g}_{t-3} + \dots)$$

- 采用动量技术后迭代轨迹会更平滑，并加快收敛

6.5.8 梯度下降法的改进

- SGD with momentum
 - 迭代序列更平滑
 - 通过调整 β , 尽快逃离饱和区



Mini-Batch Gradient Descent with momentum

```
1  begin initialize: batch_size,  $\mathbf{w}$ ,  $\eta$ ,  $\theta$ ,  $r=0$ 
2      do  $r \leftarrow r + 1$  (increment epoch)
3           $k = 0$ ,  $\Delta w_{ih} = 0$ ,  $\Delta w_{hj} = 0$ 
4          do  $k \leftarrow k+1$ 
5              randomly select a sample  $\mathbf{x}^k$ 
6               $\Delta w_{hj} \leftarrow \Delta w_{hj} + \delta_j^k y_h^k$ ,  $\Delta w_{ih} \leftarrow \Delta w_{ih} + \delta_h^k x_i^k$ 
7          until  $k = \textit{batch\_size}$ 
8               $m_{hj} \leftarrow \beta m_{hj} + (1 - \beta) \Delta w_{hj}$ ,  $m_{ih} \leftarrow \beta m_{ih} + (1 - \beta) \Delta w_{ih}$ 
9               $w_{hj} \leftarrow w_{hj} + \eta m_{hj}$ ,  $w_{ih} \leftarrow w_{ih} + \eta m_{ih}$ 
9      until  $\| \nabla E(\mathbf{w}) \| < \theta$ 
10     return  $\mathbf{w}$ 
11 end
```

6.5.8 梯度下降法的改进

- Adaptive Moment Optimization (Adam)

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t \quad \text{where} \quad \mathbf{g}_t = \left. \frac{\partial E(\mathbf{w})}{\partial \mathbf{w}} \right|_{\mathbf{w}=\mathbf{w}_t}$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_t + (1 - \beta_2) \mathbf{g}_t \odot \mathbf{g}_t$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \mathbf{m}_t ./ \sqrt{\mathbf{v}_t}$$

$\odot$ and $./$ are element-wise product and element-wise division

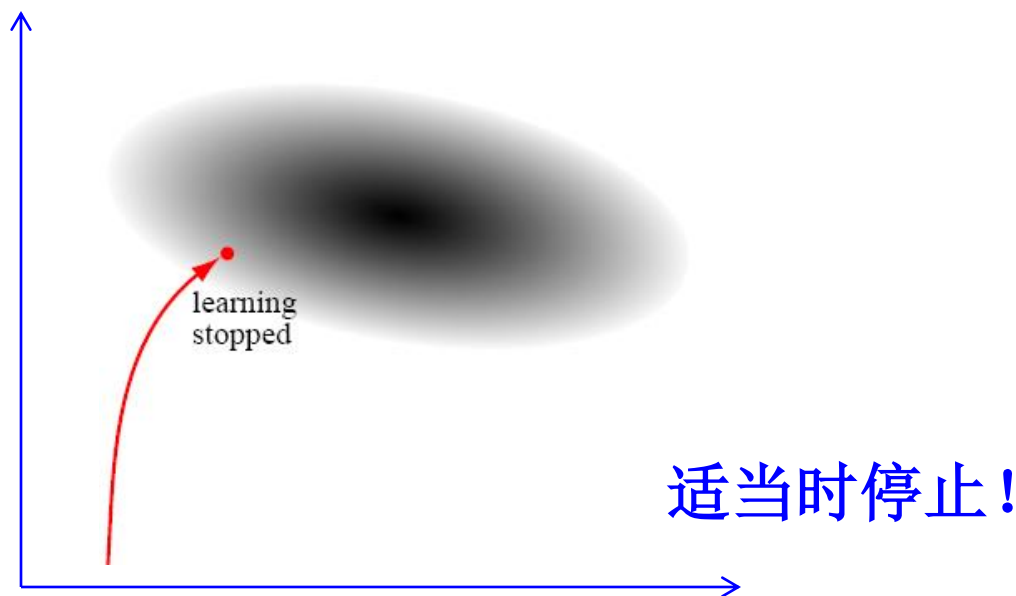
$\mathbf{m}_t$ 为一阶动量，由当前梯度和之前得到的梯度共同决定

$\mathbf{v}_t$ 为二阶动量，对每个参数使用不同的学习率

Diederik P. Kingma, Jimmy Ba. Adam: A Method for Stochastic Optimization. ICLR, 2014.

6.5.9 训练停止准则

- 训练停止—没有固定准则
 - 如果过度训练网络（即使训练样本都能正确的分类）有可能会产生 **overfitting 问题**：
 - 训练样本的分类正确率很高，但对新样本的分类能力不高。因此，网络的泛化能力不强

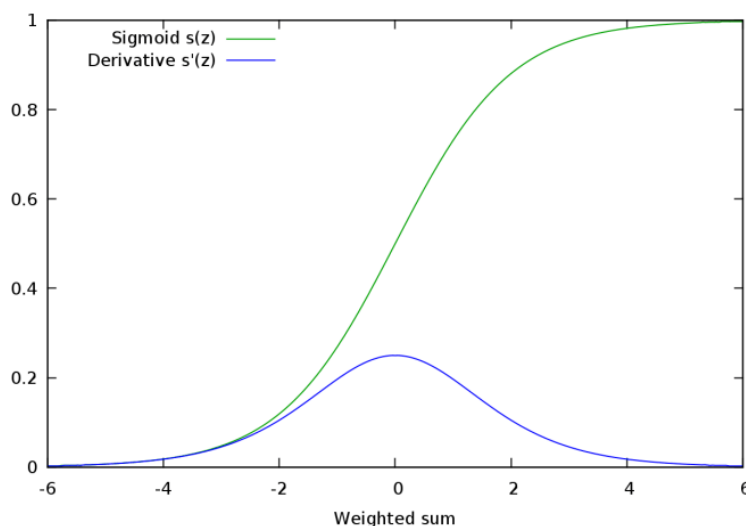


6.5.10 典型问题

- BP训练算法存在的问题
 - 尽管BP训练算法应用得很广泛，但其训练过程存在不确定性：
 - 完全难以训练
 - 网络的麻痹现象
 - 梯度消失
 - 局部最小和鞍点
 - 训练时间过长
 - 尤其对复杂问题需要很长时间训练；
 - 可能选取了不恰当的训练速率 η 。

网络的麻痹现象

- 在计算权重修正量时，误差 δ 正比于 $f'(net)$ 。
- 当 $f'(net) \rightarrow 0$ 时， $\delta \rightarrow 0$ ，从而 $\Delta w_{ij} \rightarrow 0$ ，相当于调节过程几乎停顿下来。
 - 在训练过程中（如采用Sigmoid函数），权重调得较大，可能使所有或部分加权和 net 较大，梯度更新将在S型函数的饱和区域进行，即处在其导数 $f'(net)$ 非常小的区域内。

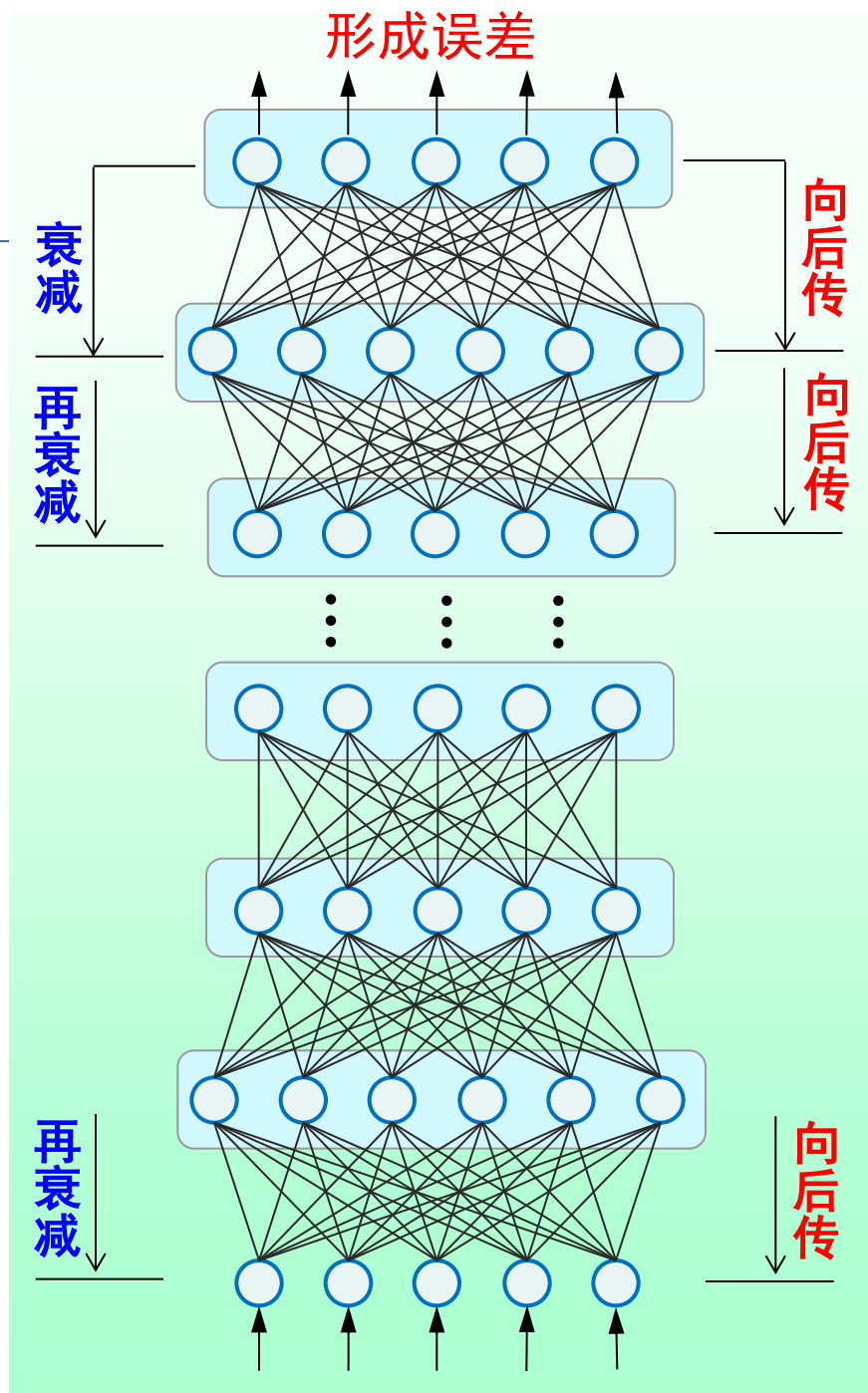


梯度消失

- 在多层前向神经网络中，越靠近输入层越容易出现此问题。
- 该问题是由于如下局部梯度在传播过程引起的：

$$\delta_h^k = f'(net_h^k) \sum_j w_{hj} \delta_j^k$$

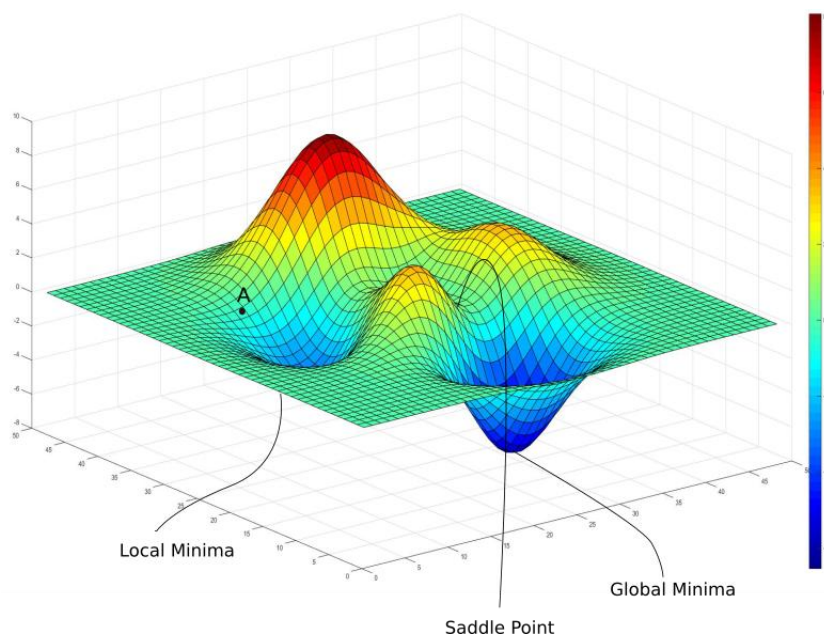
该导数通常小于1



局部极小和鞍点



- 损失函数 $E(\mathbf{w})$ 是高度非线性的复杂函数，其曲面在多维空间中凹凸不平，可能存在大量满足 $\nabla E(\mathbf{w}) = 0$ 的点
- BP算法实际上是梯度下降法，从某一起始点沿损失函数的斜面最陡方向搜索满足 $\nabla E(\mathbf{w}) = 0$ 的点。因而在训练过程中可能会陷入某一局部最小点。
- 解决方法：在网络训练中引入随机性
 - 使用随机梯度下降(mini batch)
 - 模拟退火：以一定概率接受使损失增加的参数更新
 - 使用不同初始参数多次训练



Internal Covariate Shift (ICS)

- **ICS:** 每一层的参数更新会导致上层的输入数据分布发生变化, 通过层层叠加, 高层的输入分布变化会非常剧烈, 使得高层需要不断重新适应底层的参数更新
- Batch Normalization (BN): 将数据的每维特征调整至均值为0, 方差为1

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1...m}\}$;
Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

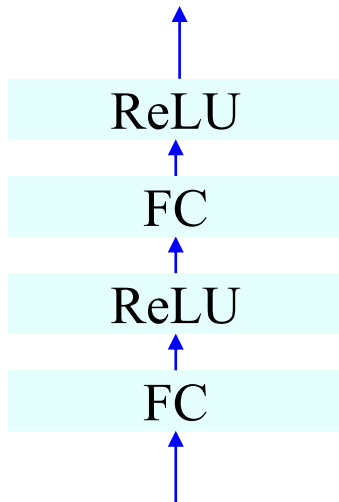
1. 使用mini-batch中的数据, 估计每维特征均值 μ 和方差 σ^2
2. 使用均值方差对该特征进行规范化
3. 对规范化后的特征重新缩放和平移



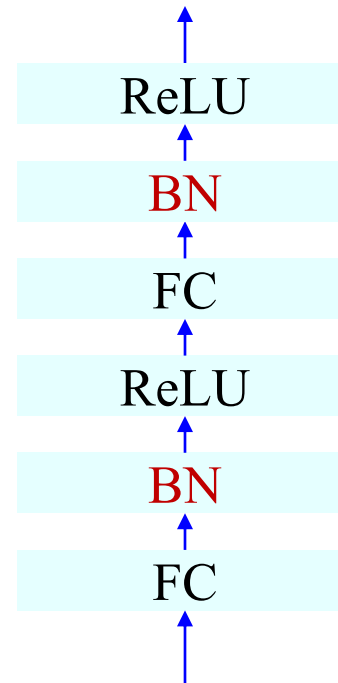
Internal Covariate Shift (ICS)

- Batch Normalization (BN)
 - 在输入非线性激活函数前，对数据特征使用BN操作

FC表示全
连接层



$$\mathbf{z} = \text{ReLU}(\mathbf{W}\mathbf{x} + \mathbf{b})$$



$$\mathbf{z} = \text{ReLU}(\text{BN}(\mathbf{W}\mathbf{x}))$$

Batch normalization, Layer normalization等规范化操作对深层网络的训练具有重要作用

6.6 径向基函数网络

- Radial Basis Function networks (RBF networks)
 - 是一种三层的神经网络，使用径向基函数作为隐层计算单元
 - 与多层感知器相似，RBF网络可以对任意连续的非线性函数进行近似，可以处理数据中的难以解析的规律性
 - 收敛速度比通常的多层神经网络更快
 - 对模式分类或函数近似任务，径向基函数是一个较好的网络模型

D. Hush and B. Horne, Progress in Supervised Neural Networks, IEEE Signal Processing Magazine, 8-39, 1993.

6.6 径向基函数网络

- RBF networks

- 使用径向基函数 $\{\phi_h(\mathbf{x})\}$ 的线性组合表示每类的判别函数：

$$z_j = \sum_{h=1}^H w_{hj} \phi_h(\mathbf{x}), \quad j = 1, 2, \dots, c$$

其中， $\phi_h(\mathbf{x})$ 为径向基函数，如 $\phi_h(\mathbf{x}) = \exp\left(-\frac{\|\mathbf{x} - \mathbf{m}_h\|^2}{2\sigma^2}\right)$

- $\mathbf{m}_h$ ：径向基函数的中心
- $\phi_h(\mathbf{x})$ ： $\mathbf{x}$ 到 $\mathbf{m}_h$ 之间距离的单调减函数，表示 $\mathbf{x}$ 与 $\mathbf{m}_h$ 的相似度

6.6 径向基函数网络

- 径向基函数

- 径向基函数通常采用高斯函数，也可采用其它函数。

$$\phi(\mathbf{x}) = \exp\left(-\frac{\|\mathbf{x} - \mathbf{m}\|^2}{2\sigma^2}\right) \quad (\text{Gaussian})$$

$$\phi(\mathbf{x}) = \frac{1}{1 + \exp\left(\frac{\|\mathbf{x} - \mathbf{m}\|^2}{2\sigma^2}\right)} \quad (\text{Reflected Sigmoidal})$$

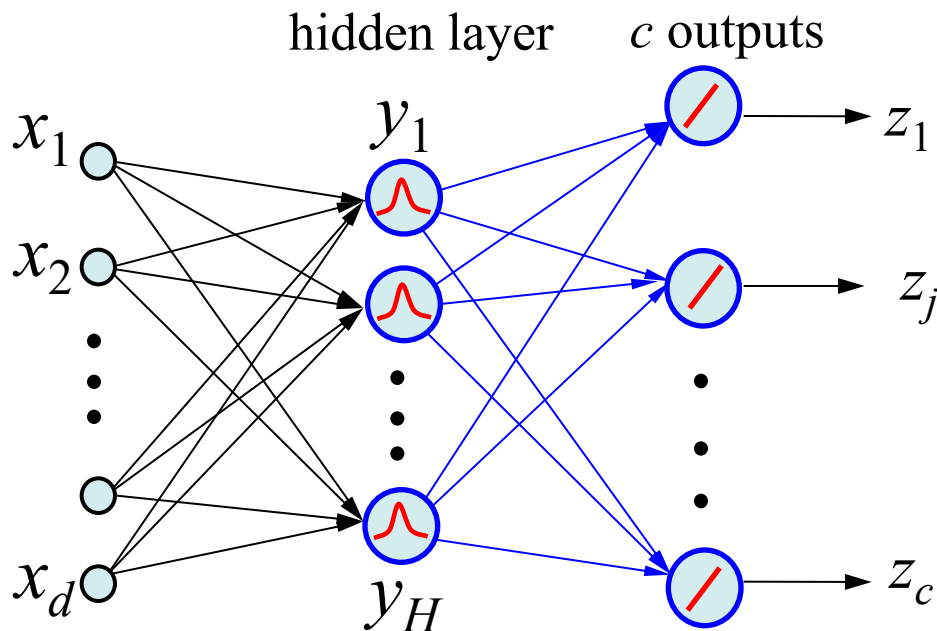
$$\phi(\mathbf{x}) = \frac{1}{\sqrt{\|\mathbf{x} - \mathbf{m}\|^2 + \sigma^2}} \quad (\text{Inverse multi-quadrics})$$

m为径向基函数的中心

6.6 径向基函数网络

- 网络结构

- 一个三层神经网络，**隐含层激活函数为高斯函数**，隐含层结点数为RBF个数。



RBF $\phi_h(x)$ 的中心定义了输入层到隐层节点 h 的权重

$$net_h = \sum_{i=1}^d (x_i - w_{ih})^2$$

$$y_h = \exp\left(-\frac{net_h}{2\sigma^2}\right)$$

$$z_j = \sum_h w_{hj} y_h$$

6.6 径向基函数网络

- RBF网络的学习

$$z_j = \sum_{h=1}^H w_{hj} \phi_h(\mathbf{x}), \quad j = 1, 2, \dots, c$$

- 通常采用两步方法训练

- (1) 确定径向基函数的个数、中心

- 小规模数据：用全部样本

- 大规模数据：采样、聚类

- (2) 确定隐层到输出层的权重、高斯核窗宽

- BP等算法

- 端到端方法：使用BP算法学习径向基函数中心、窗宽、隐层到输出层的权重

6.6 径向基函数网络

- 网络结构简化与普遍化
 - 对于大规模数据，隐含层的结点个数会很大。通常，对数据进行聚类，**将聚类中心作为径向基函数的中心。这样简化了网络规模，提高计算效率。**
 - 经过聚类处理，还会防止 overfitting，增强网络的泛化能力，提高精度。
 - **输出层结点也可以采用非线性激活函数**

6.6 径向基函数网络

- RBF网络的学习

第一个样本的
RBF特征表示

第一类判别函
数的待求权重

第一个样本的
目标向量

$$\underbrace{\begin{pmatrix} \phi_1(\mathbf{x}_1) & \phi_2(\mathbf{x}_1) & \cdots & \phi_H(\mathbf{x}_1) \\ \phi_1(\mathbf{x}_2) & \phi_2(\mathbf{x}_2) & \cdots & \phi_H(\mathbf{x}_2) \\ \vdots & \vdots & & \vdots \\ \phi_1(\mathbf{x}_n) & \phi_2(\mathbf{x}_n) & \cdots & \phi_H(\mathbf{x}_n) \end{pmatrix}}_{\Phi} \underbrace{\begin{pmatrix} w_{11} & w_{12} & \cdots & w_{1c} \\ w_{21} & w_{22} & \cdots & w_{2c} \\ \vdots & \vdots & & \vdots \\ w_{H1} & w_{H2} & \cdots & w_{Hc} \end{pmatrix}}_{\mathbf{W}} = \underbrace{\begin{pmatrix} t_1^1 & t_2^1 & \cdots & t_c^1 \\ t_1^2 & t_2^2 & \cdots & t_c^2 \\ \vdots & \vdots & & \vdots \\ t_1^n & t_2^n & \vdots & t_c^n \end{pmatrix}}_{\mathbf{T}}$$

$$\Phi \mathbf{W} = \mathbf{T} \quad \text{or} \quad \min \|\mathbf{T} - \Phi \mathbf{W}\|^2$$

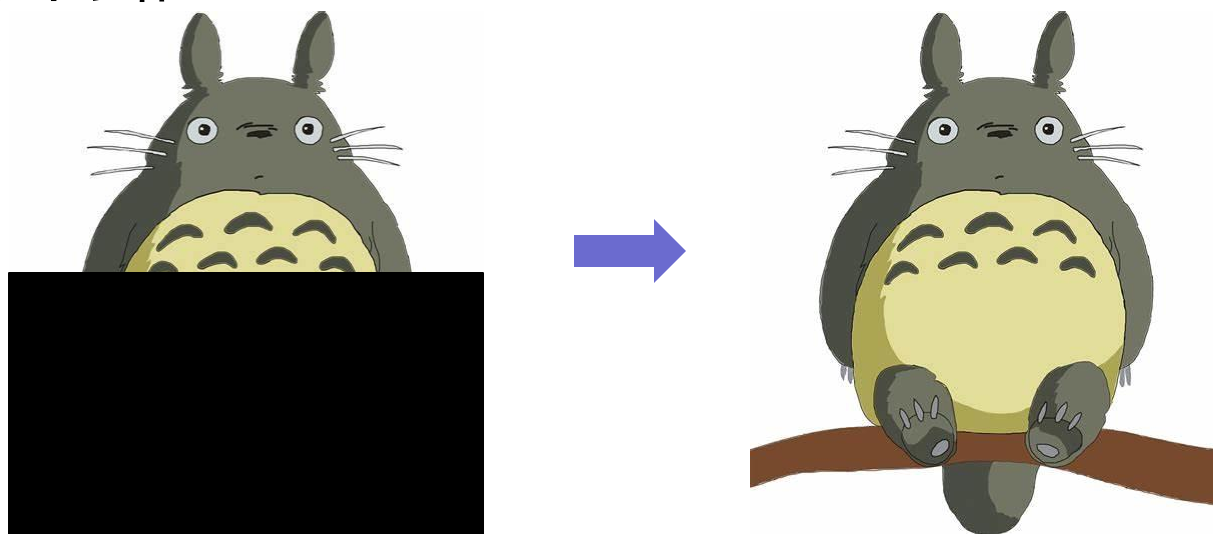
6.7 反馈神经网络

- 按照神经网络运行过程中的信息流向分类：
 - 前馈网络
 - 通过许多具有简单处理能力的神经元的复合作用，使整个网络具有复杂的非线性映射能力。
 - 反馈网络
 - 通过网络神经元状态的变迁，最终稳定于某一状态，得到联想存储或神经计算的结果。
 - 典型的反馈神经网络
 - Hopfield网络、Boltzman机、受限Boltzman机
 - 反馈网络具有一般非线性系统的许多性质，如稳定性、各种类型的吸引子以及混沌现象，等等。

6.7.1 Hopfield网络 (Hopfield, 1982)

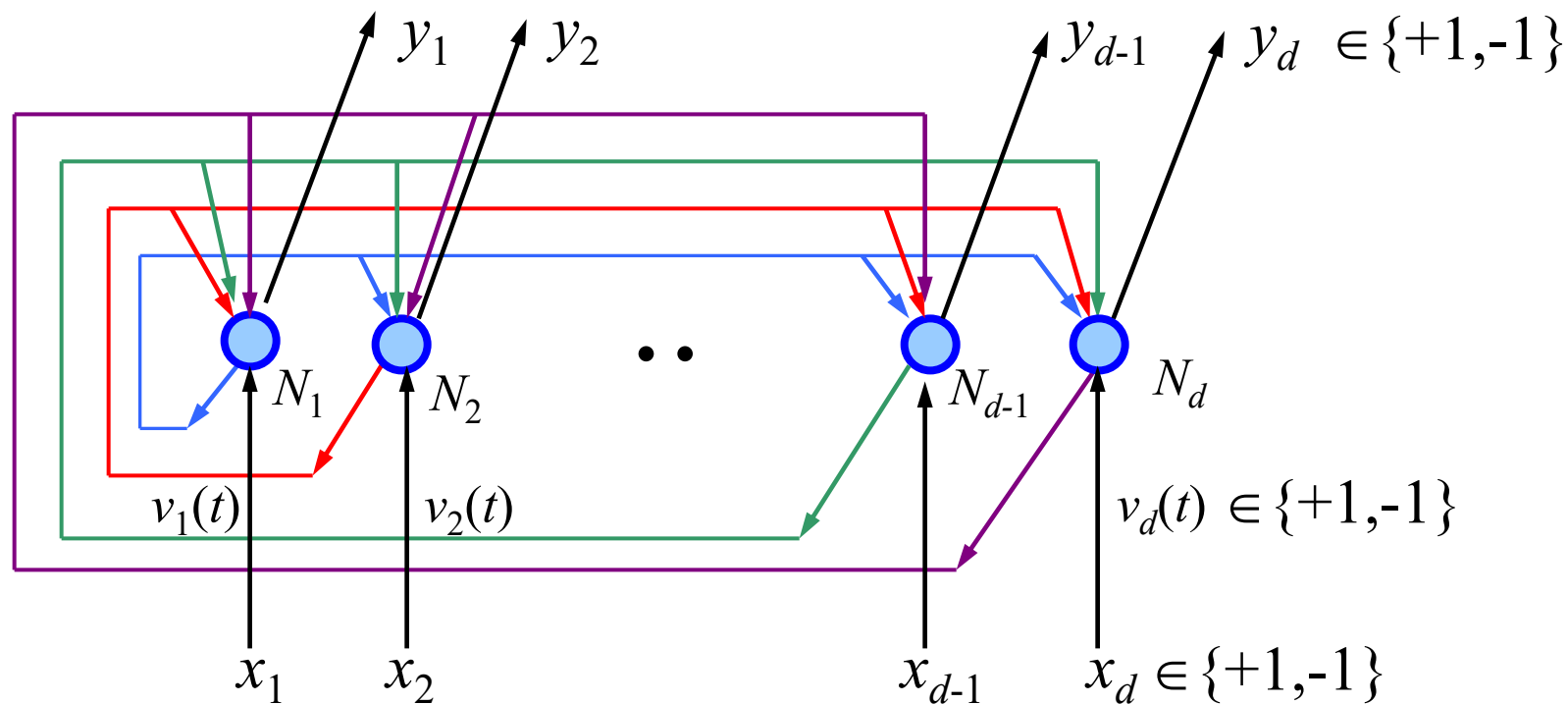
- 网络描述

- Hopfield网络是一种单层、全连接的反馈网络
- 它的目的是从残缺或噪声数据中还原完整数据；“基于部分数据提取完整数据”与人的联想记忆(associative memory)有相似之处，因此，Hopfield网络也称为联想记忆网络。



6.7.1 Hopfield网络

- 离散型Hopfield网络结构



单层，全连接，且各结点地位相同

6.7.1 Hopfield网络

$$\text{sgn}(x) = \begin{cases} -1 & x < 0 \\ 1 & x \geq 0 \end{cases}$$

设网络有 d 个神经元，对于离散型Hopfield网络，各结点一般选取相同的激活函数，如符号函数，即： $f_1(x) = f_2(x) = \dots = f_d(x) = \text{sgn}(x)$ 。

- 网络的输入： $\mathbf{x} = (x_1, x_2, \dots, x_d)$, $\mathbf{x} \in \{-1, +1\}^d$
- 网络的输出： $\mathbf{y} = (y_1, y_2, \dots, y_d)$, $\mathbf{y} \in \{-1, +1\}^d$
- 网络在时刻 t 的状态： $\mathbf{v}(t) = (v_1(t), v_2(t), \dots, v_d(t))$, $\mathbf{v}(t) \in \{-1, +1\}^d$

其中， t 为离散时间变量。

- 连接权 w_{ij} ： $w_{ij} = w_{ji}$, $w_{ii} = 0$, $i, j \in \{1, 2, \dots, d\}$

Hopfield网络是对称、无自环的

网络所有 d 个结点之间的连接权用矩阵 $\mathbf{W} = \{w_{ij}\} \in R^{d \times d}$ 来表示。

6.7.1 Hopfield网络

- 网络运行方式：信息在网络中循环往复传递
 - 运行过程（联想阶段）：

$$\begin{cases} v_j(0) = x_j & x_j: \text{输入信号}\mathbf{x}\text{的第}j\text{个分量} \\ v_j(t+1) = \text{sgn}\left(\sum_{i=1}^d w_{ij} v_i(t)\right) & \Leftrightarrow \mathbf{v}(t+1) = \text{sgn}(\mathbf{W}\mathbf{v}(t)) \end{cases}$$

- 若有某个时刻 t ，从此之后网络状态不再改变，即：
 $\mathbf{v}(t+1) = \mathbf{v}(t)$ ，则输出 $\mathbf{y} = \mathbf{v}(t)$ 。

稳定条件： $\mathbf{v} = \text{sgn}(\mathbf{W}\mathbf{v})$

6.7.1 Hopfield网络

- 网络运行方式：信息在网络中循环往复传递
 - 运行过程（联想阶段）：

$$\begin{cases} v_j(0) = x_j & x_j: \text{输入信号}\mathbf{x}\text{的第}j\text{个分量} \\ v_j(t+1) = \text{sgn}\left(\sum_{i=1}^d w_{ij} v_i(t)\right) & \Leftrightarrow \mathbf{v}(t+1) = \text{sgn}(\mathbf{W}\mathbf{v}(t)) \end{cases}$$

- Hopfield网络的运行过程为状态的演化过程，即从初始状态 $\mathbf{v}_0$ 按能量减小的方向进行演化，直到达到稳定状态。

$$\min_{\mathbf{v} \in \{-1,1\}^d} E(\mathbf{v}) = -\frac{1}{2} \sum_{i,j=1}^d w_{ij} v_i v_j = -\frac{1}{2} \mathbf{v}^T \mathbf{W} \mathbf{v}$$

将残缺数据 $\mathbf{v}_0=\mathbf{x}$ 输入网络，网络通过最小化能量函数 $E(\mathbf{v})$ 恢复完整数据

6.7.1 Hopfield网络

- Hopfield网络的学习
 - 参数估计（记忆阶段）：将完整数据存储进网络
 - 若网络训练的目的在于存储 K 个完整数据

$$\{\mathbf{x}^1, \mathbf{x}^2, \dots, \mathbf{x}^K\} \text{ where } \mathbf{x}^k \in \{-1, 1\}^d$$

通常要求 $K < 0.14d$

- 外积法：

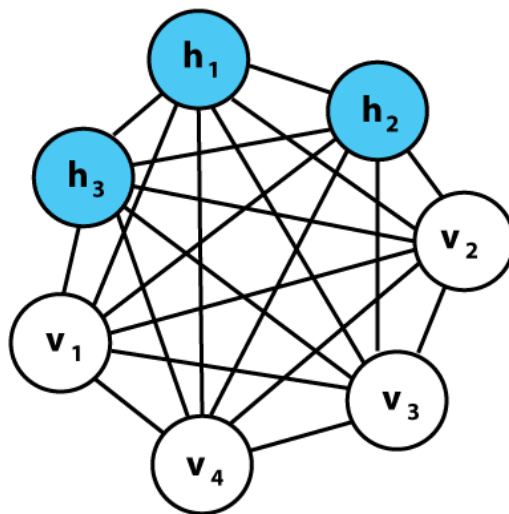
$$w_{ij} = \begin{cases} \sum_{k=1}^K x_i^k x_j^k, & i \neq j \\ 0, & i = j \end{cases} \Leftrightarrow \mathbf{W} = \sum_{k=1}^K \mathbf{x}^k (\mathbf{x}^k)^T - K\mathbf{I}$$

$\mathbf{I} \in R^{d \times d}$ 是单位矩阵

Hebbian learning rule

6.7.2 玻尔兹曼机 (Hinton & Sejnowski, 1985)

- Boltzmann Machine, BM
 - BM是一种**随机的**Hopfield网络，是具有隐单元的反馈型网络
 - 网络结构
 - BM中的神经元是随机的、二值的
 - 所有的神经元之间都是对称连接的，没有自连接
 - **BM中一部分神经元与外部相连，起到网络的输入、输出功能。**另一部分神经元则不与外部相连，属于隐单元



$$w_{ji} = w_{ij}, w_{ii} = 0$$

对称，无自连

6.7.2 玻尔兹曼机

- Boltzmann Machine, BM

- BM的能量函数

$$E(\mathbf{v}, \mathbf{h}) = -\sum_{ij} W_{ij} v_i h_j - \sum_{ik} U_{ik} v_i v_k - \sum_{jl} V_{jl} h_j h_l - \sum_i b_i v_i - \sum_j c_j h_j$$

- $\{b_i\}, \{c_i\}$ 是可见层和隐层的bias
- $\{W_{ij}\}$ 是可见单元与隐含单元之间的连接权重
- $\{U_{ij}\}$ 是可见单元之间的连接权重
- $\{V_{ij}\}$ 是隐含单元之间的连接权重

- 玻尔兹曼分布

$$p_{\theta}(\mathbf{v}, \mathbf{h}) = \frac{1}{Z(\theta)} \exp(-E(\mathbf{v}, \mathbf{h}; \theta))$$

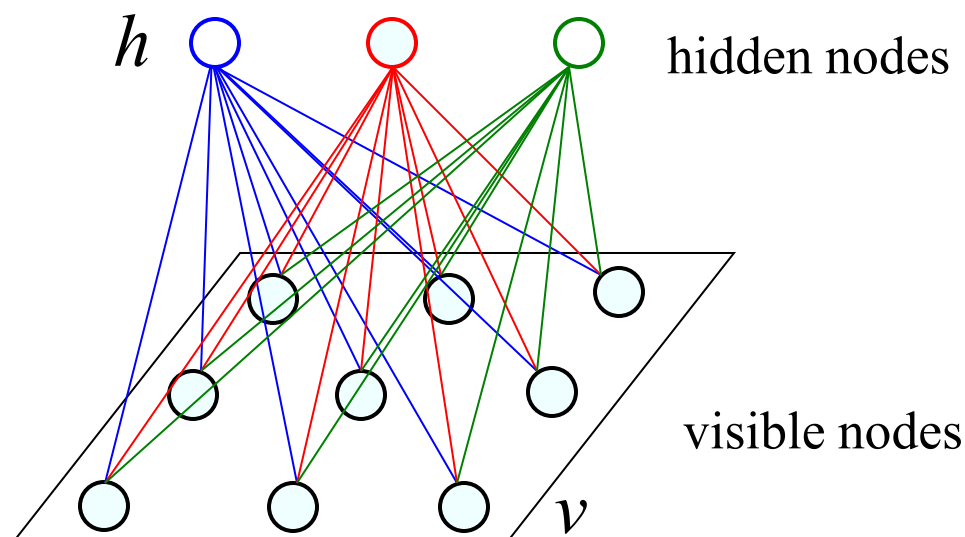
其中, $\theta = \{W, U, V, b, c\}$, $Z(\theta)$ 为配分函数(partition function)

6.7.2 玻尔兹曼机

- Boltzmann Machine, BM
 - BM的采样 (inference)
 - 吉布斯采样
 - 随机选择一个单元，采样更新状态，反复迭代达到热平衡
 - BM的训练 (learning)
 - 最大化样本的观测概率（即最小化样本的能量）
$$\max_{\theta} \sum_{k=1}^n \log P(\mathbf{v}^k | \theta)$$
 - 但梯度的计算需要复杂的采样过程，无法在大规模问题上应用

6.7.3 受限玻尔兹曼机

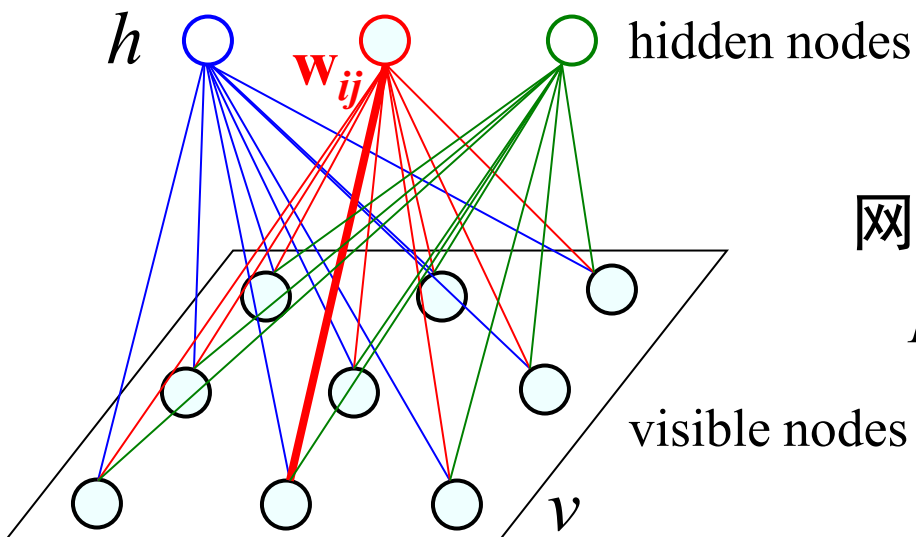
- Restricted BM, RBM
 - 是一种简化了的玻尔兹曼机
 - 具有两层结构，层间相连，层内结点不相连，信息可双向流动
 - 包含可视结点层（与外界相连）和隐含层



RBM

Classical structure

Stochastic binary visible variables $\mathbf{v} \in \{0,1\}^d$ are connected to stochastic binary hidden variables $\mathbf{h} \in \{0,1\}^m$.



网络的能量函数：

$$E(\mathbf{v}, \mathbf{h}; \theta) = -\sum_{ij} w_{ij} v_i h_j - \sum_i b_i v_i - \sum_j c_j h_j$$

$\theta = \{w, b, c\}$ —模型参数

可见状态和隐含状态的联合概率分布：

$$p_{\theta}(\mathbf{v}, \mathbf{h}) = \frac{1}{Z(\theta)} \exp(-E(\mathbf{v}, \mathbf{h}; \theta)) = \frac{1}{Z(\theta)} \prod_{ij} e^{w_{ij} v_i h_j} \prod_i e^{b_i v_i} \prod_j e^{c_j h_j}$$

$$Z(\theta) = \sum_{\mathbf{v}, \mathbf{h}} \exp(-E(\mathbf{v}, \mathbf{h}; \theta))$$

玻尔兹曼分布

6.7.3 受限玻尔兹曼机

- RBM的采样
 - RBM中的条件独立性
 - 因为没有同层连接，可见层条件独立，隐层条件独立
 - RBM中的条件概率

可见层给定下的隐层: $P(h_j = 1 | \mathbf{v}) = \sigma(c_j + \sum_i W_{ij} v_i)$

隐层给定下的可见层: $P(v_i = 1 | \mathbf{h}) = \sigma(b_i + \sum_j W_{ij} h_j)$

- 采样

- 因为条件概率均为条件独立的Bernoulli分布，RBM 使用交替 Gibbs sampling

Given $\mathbf{v}$:

sample each h_j independently from $P(h_j = 1 | \mathbf{v})$

Given $\mathbf{h}$:

sample each v_i independently from $P(v_i = 1 | \mathbf{h})$

重复多次会趋近 RBM 的平稳分布 (Boltzmann 分布)

6.7.3 受限玻尔兹曼机

- RBM的训练

- 给定 N 个样本: $\max_{\theta} \sum_{k=1}^N \log p_{\theta}(\mathbf{v}^k)$

$$\log(p_{\theta}(\mathbf{v})) = \log \sum_{\mathbf{h}} p_{\theta}(\mathbf{v}, \mathbf{h}) = \log \left(\prod_i \exp(b_i v_i) \prod_j \left(1 + \exp(c_j + \sum_i w_{ij} v_i) \right) \right) - \log Z(\theta)$$

- 参数梯度: $\frac{\partial P(\mathbf{v})}{\partial w_{ij}} = \underbrace{\langle v_i h_j \rangle_{\text{data}}}_{v_i h_j \text{ 在训练数据下的期望}} - \underbrace{\langle v_i h_j \rangle_{\text{model}}}_{v_i h_j \text{ 在模型分布下的期望}}$

- 难点: 模型期望需要从模型分布中采样, 但收敛缓慢
 - 改进方法: Contrastive Divergence算法(使用近似采样)

6.8 自组织映射

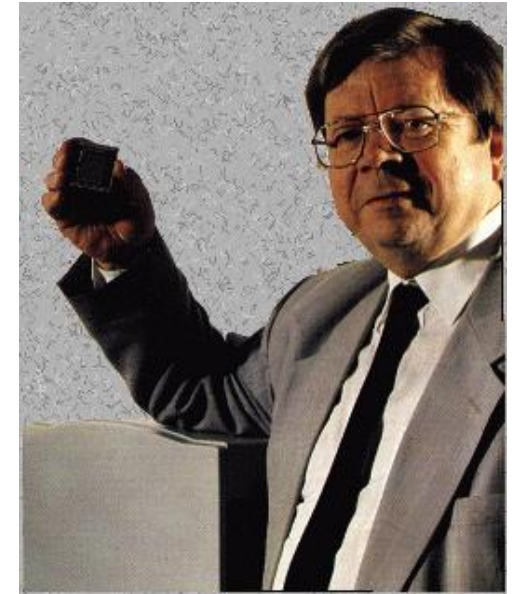
- 自组织竞争神经网络类型
 - 自组织映射(Self-Organizing Map, SOM)网络
 - 自适应共振理论(Adaptive Resonance Theory, ART)网络
 - 对传(Counter Propagation, CP)网络
 - 协同神经网络(Synergetic Neural Network, SNN)

6.8 自组织映射 (Self-Organizing Maps, SOM)

- 描述

T. Kohonen认为：神经网络中邻近的各个神经元通过侧向交互作用彼此竞争，自适应地发展成检测不同信号的特殊检测器。

Kohonen的思想在本质上是希望解决有关外界信息在人脑中自组织地形成概念的问题。



芬兰：T. Kohonen (1981)

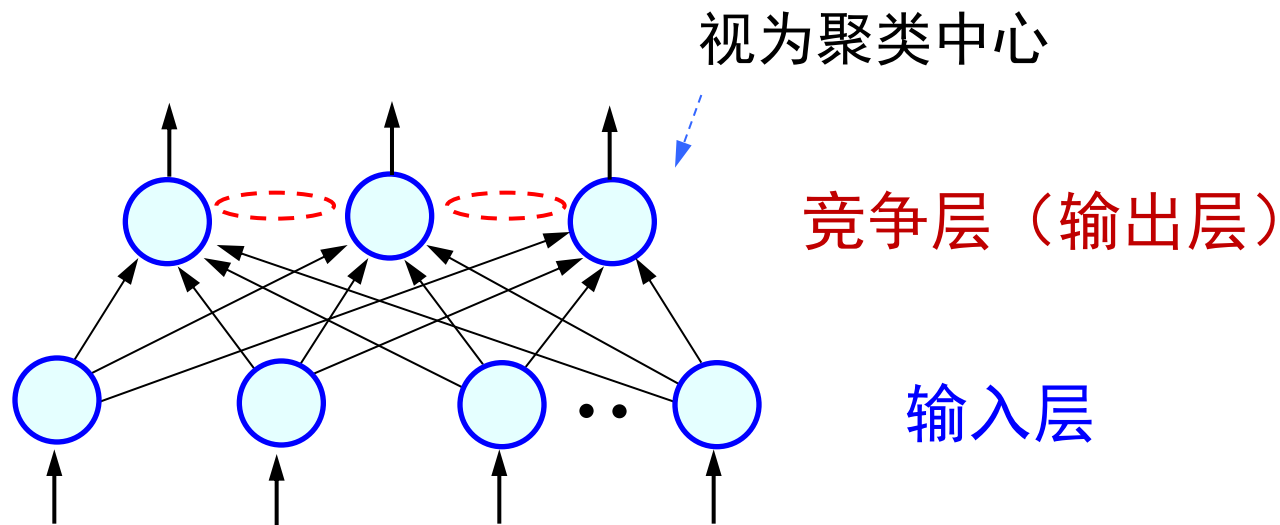
6.8 自组织映射

- 生物学基础

- ✓ 在人脑的感知通道上，神经元是**有序排列的**：特定的输入能够激活特定的神经元；**且相似的输入所激活的神经元位置上是相近的**。（连续映像、拓扑保持）
- ✓ 在生物神经系统中，存在**侧兴奋/侧抑制**现象：**一个神经元兴奋后，会对其近邻的神经元产生刺激作用，对距其较远的神经元产生抑制作用**。这种相互作用方式使神经元细胞之间产生竞争，使一些获胜，一些失败。（竞争学习）

6.8 自组织映射

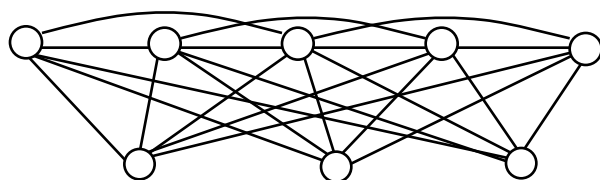
- 两层结构



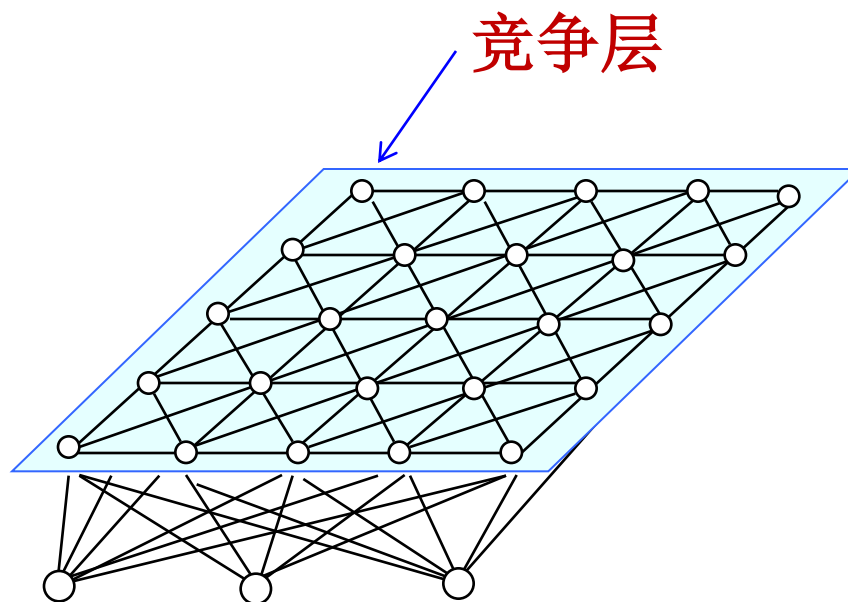
与两层前馈神经网络相似，输入层的每一个单元与输出层的每个单元相联，且具有权重。但输出层相邻神经元之间存在相互作用。

6.8 自组织映射

- 常见的SOM网络结构



一维线结构



二维平面结构

注：竞争层结点之间的连线表示近邻间的相互作用，但没有信息传递

6.8 自组织映射

- SOM网络的前向推理

- Kohonen自组织特征映射算法，能够自动找出输入样本之间的相似度，将相似的输入在网络上就近配置。

- 相似度准则：

- 欧氏距离：
$$d_j = \sqrt{\sum_{i=1}^d (x_i - w_{ij})^2}$$
 $w_j \in R^d$ 可看作是第 j 个聚类中心

- 网络输出：

$$z_j = \begin{cases} 1 & j = j^* \\ 0 & j \neq j^* \end{cases}$$

$j^* = \arg \min_j d_j$ 为当前输入的获胜单元

6.8 自组织映射

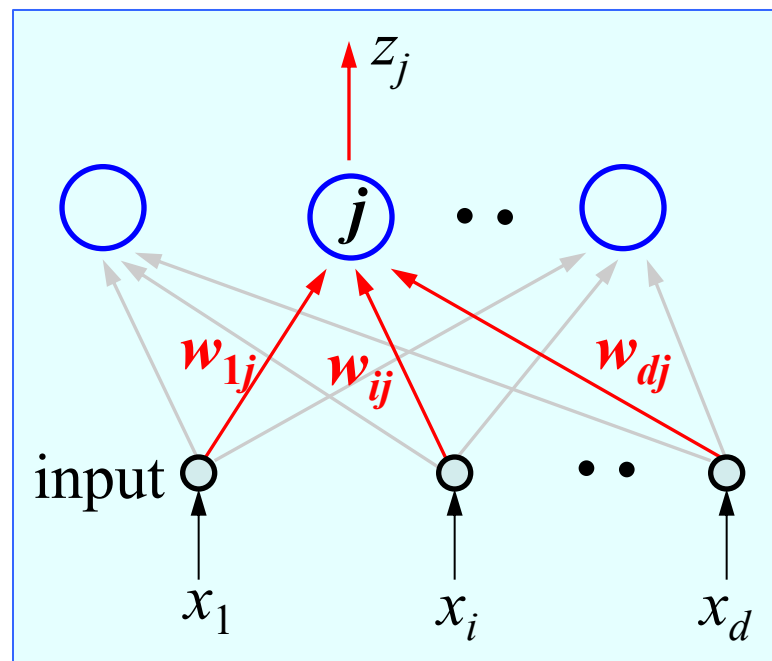
- SOM学习算法的原理
 - 通过自动寻找样本中的内在规律和本质属性，自组织、自适应地改变网络参数与结构。（无监督学习）
 - SOM的自组织功能是通过竞争学习来实现的。
 - 竞争学习规则
 - 网络的输出神经元之间相互竞争并期望被激活；
 - 对每个输入样本 $\mathbf{x}$ ，只有一个输出神经元被激活；
 - 被激活的神经元称为获胜神经元，获胜神经元对其它神经元的影响是由近及远的，由兴奋逐渐转变为抑制。

6.8 自组织映射

- 学习步骤

- S_1 : 网络初始化—通常采用随机初始化方法
- S_2 : 输入向量 $\mathbf{x}$
- S_3 : 计算竞争层的权重向量到 $\mathbf{x}$ 的距离:

$$d_j = \sqrt{\sum_{i=1}^d (x_i - w_{ij})^2}$$



- 学习步骤

- S_4 : 选择与 $\mathbf{x}$ 距离最小的神经元，将其作为获胜神经元(j^*)，并给出其近邻神经元集合。
- S_5 : 调整权重
 - 获胜神经元和其近邻神经元的权重，按下式更新：

$$\Delta w_{ij} = \eta h(j, j^*)(x_i - w_{ij}) \quad h(,) \text{为邻域函数}$$

$$w_{ij}(t+1) = w_{ij}(t) + \Delta w_{ij}$$

$$\Rightarrow \|\mathbf{w}_j(t+1) - \mathbf{x}\| = \|\mathbf{w}_j(t) + \eta h(j, j^*)(\mathbf{x} - \mathbf{w}_j(t)) - \mathbf{x}\| = (1 - \eta h(j, j^*))\|\mathbf{w}_j(t) - \mathbf{x}\|$$

若 $h(j, j^*) > 0$, 神经元更靠近输入样本(兴奋)

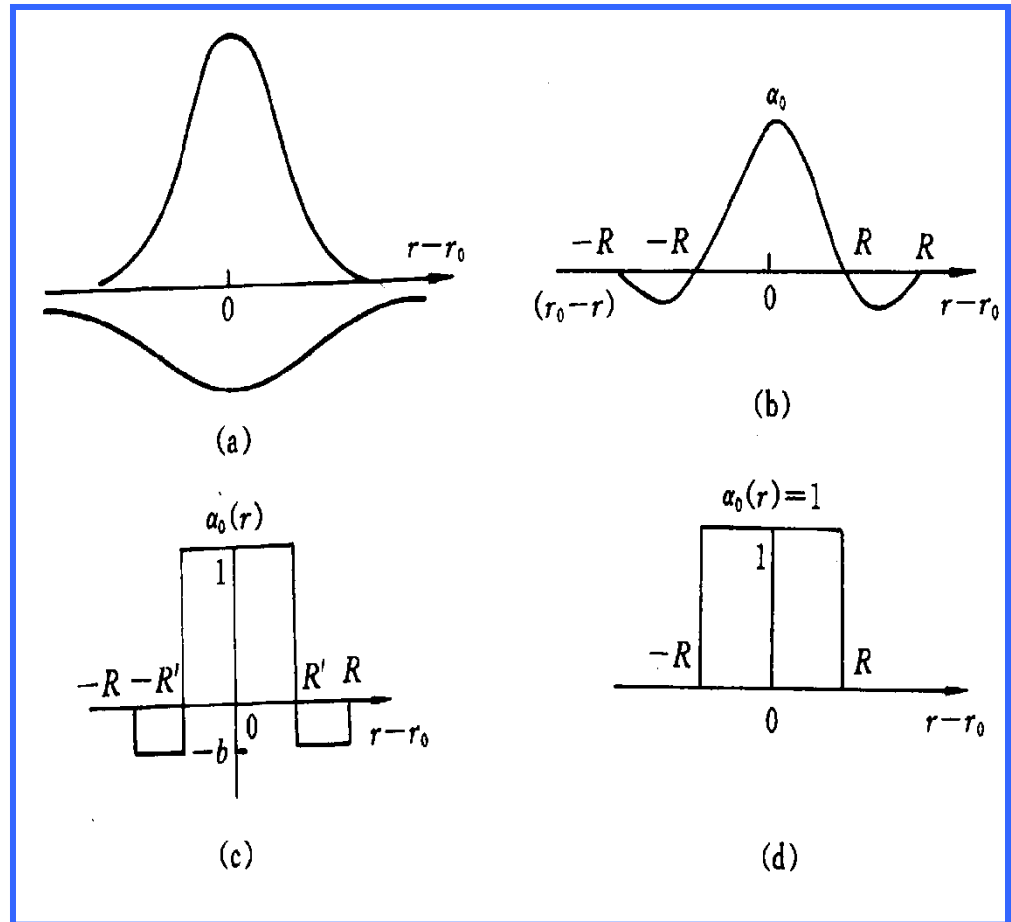
若 $h(j, j^*) < 0$, 神经元更远离输入样本(抑制)

- S_6 : 检查是否达到预先设定的要求。如达到要求则算法结束；否则返回 S_2 ，进入下一轮学习。

6.8 自组织映射

SOM获胜神经元对其它神经元的影响是由近及远的，由兴奋逐渐转变为抑制。

因此在学习算法中，不仅获胜神经元本身要调整权向量，它周围的神经元在其影响下也要不同程度地调整权重。



不同的邻域函数

6.8 自组织映射

- 邻域函数

$$h(j, j^*) = \exp\left(-\|j - j^*\|^2 / \sigma^2\right)$$

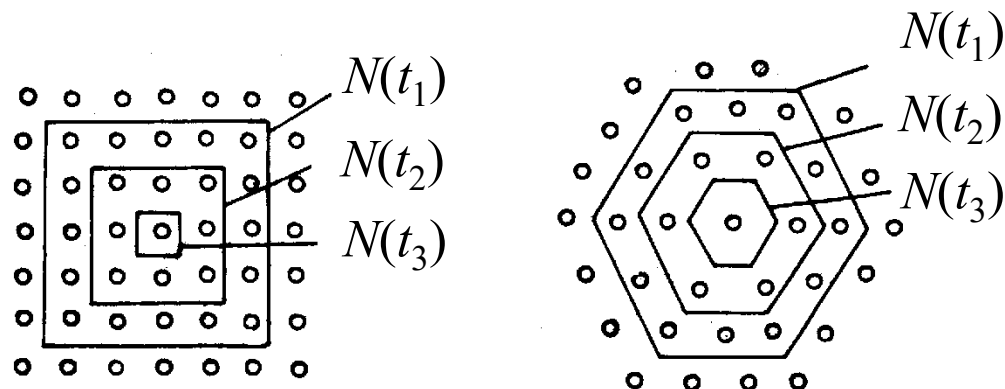
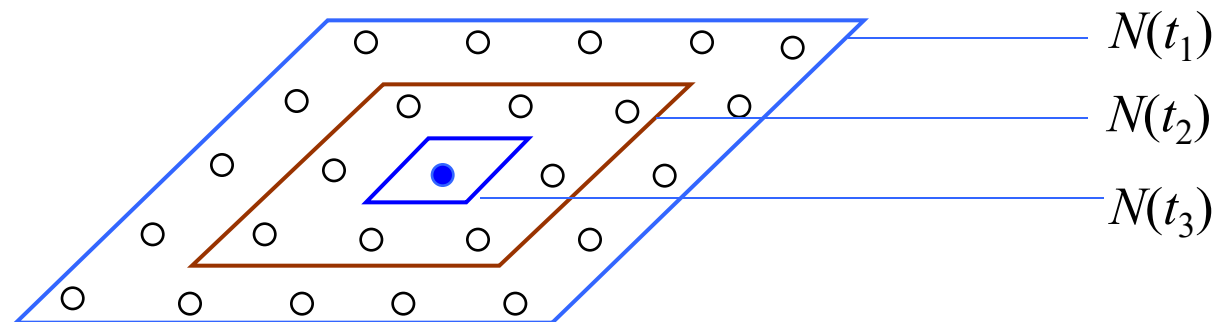
由邻域函数可以看到，以获胜神经元为中心设定了一个邻域半径，称为**获胜邻域**。

学习初期，**获胜邻域**较大，获胜神经元和其附近的神经元全部接近当前的输入向量，形成粗略的映射。

随着学习的进行而减小，**获胜邻域**变窄，获胜神经元附近的神经元数变少。因此，学习方法是一种从粗调整向微调整变化，最终达到预定目标的过程。

6.8 自组织映射

- 邻近结点相互作用
 - 权重作用的邻域大小可随训练时间增长而减小



$$t_1 < t_2 < t_3$$

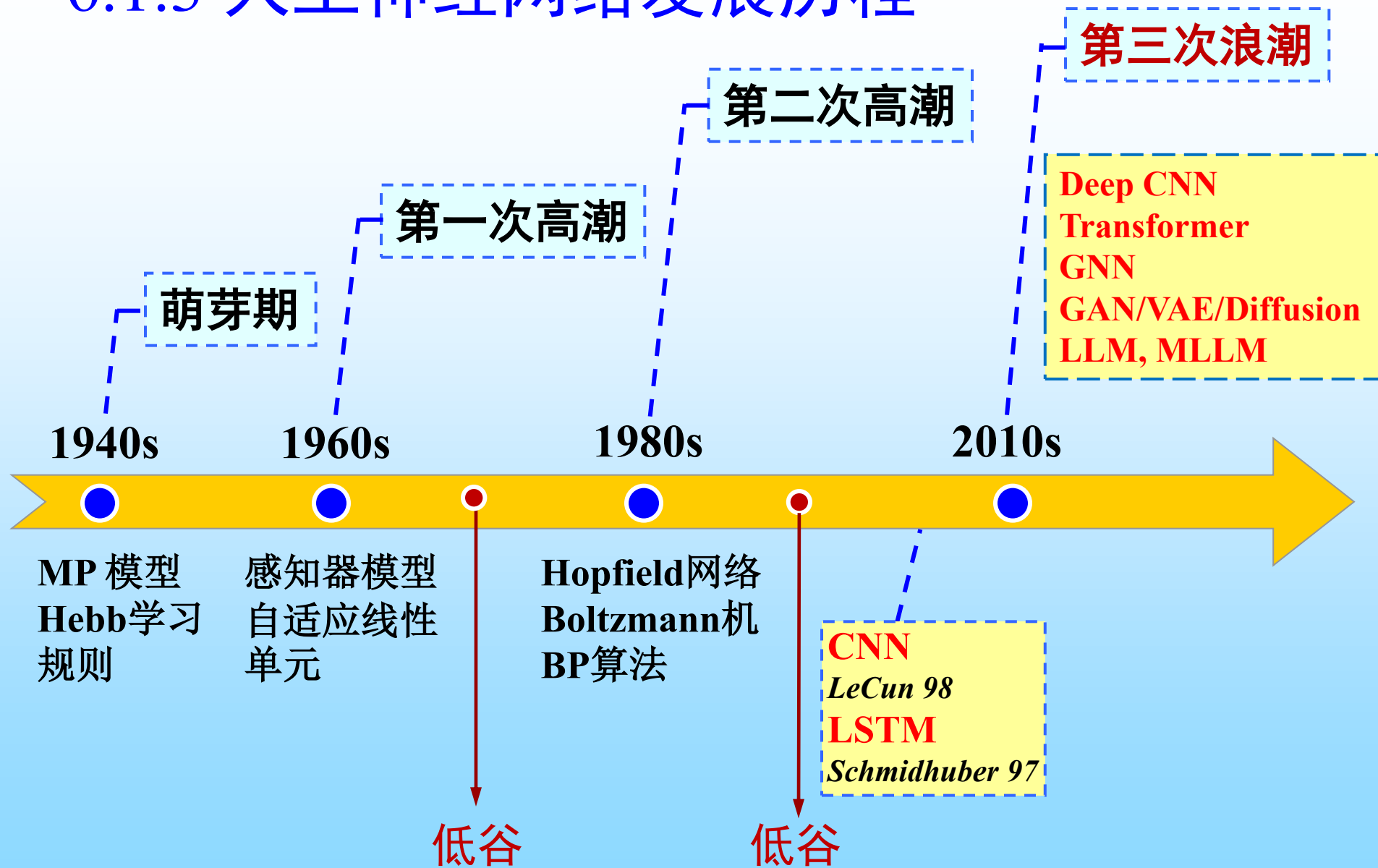
训练开始时，一般将近邻范围取得较大。
随着训练的进行其近邻范围逐渐缩小。

SOM Training Algorithm

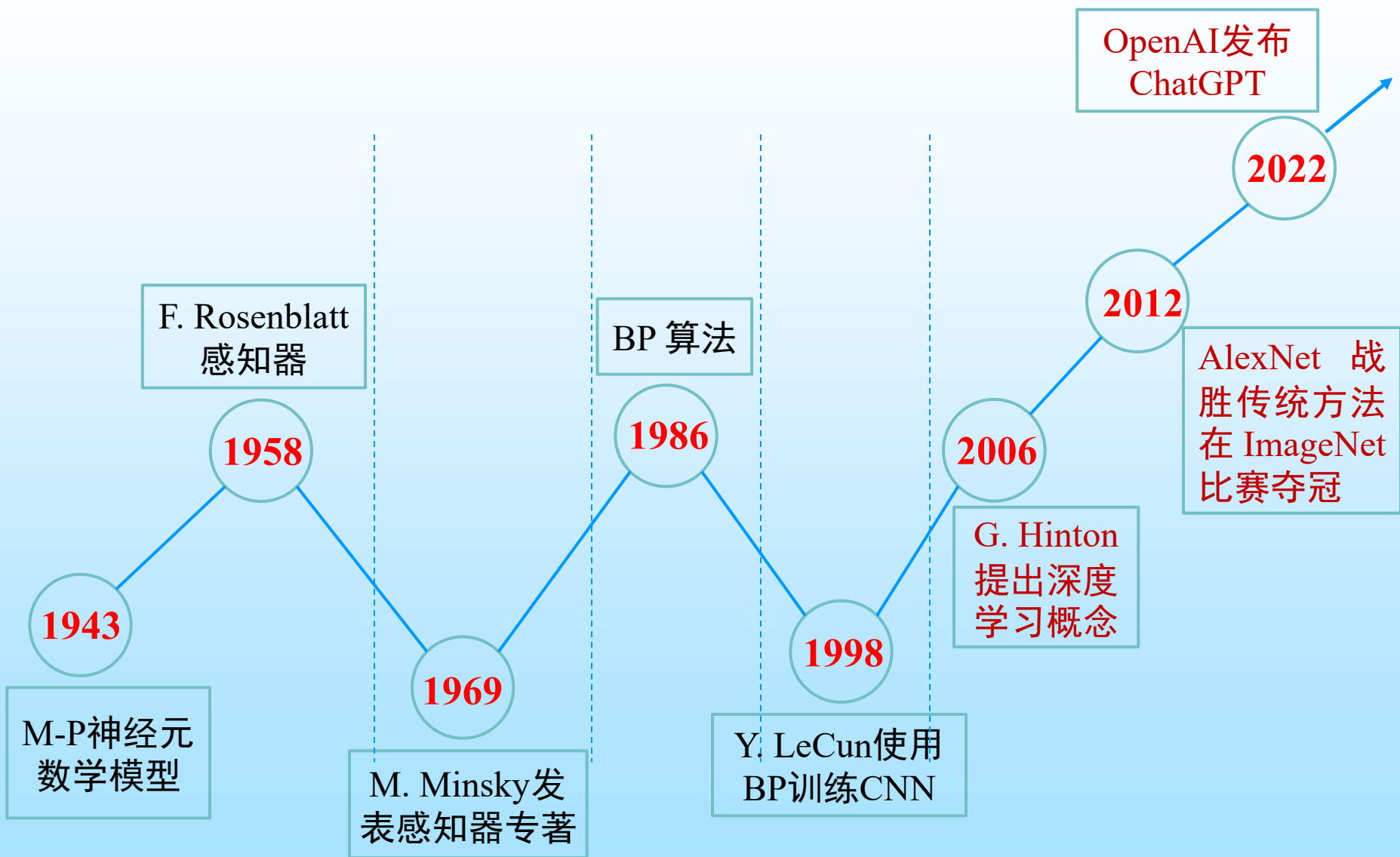
- 1 初始化、归一化权向量 $\mathbf{w}_j$ ，建立初始优胜邻域 N_j , $j=1,2,\dots,c$, 学习率 η
- 2 do $k \leftarrow k+1 \pmod n$
- 3 $\mathbf{x}^k$ randomly chosen a sample (normalized)
- 4 calculate $d_j = \|\mathbf{x}^k - \mathbf{w}_j\|$, $j=1, 2,\dots,c$,
- 5 find the index with the minimum distance, denote j^*
- 6 update the winner's neighborhood $N_j(j^*)$
- 7 adjust the weights of the nodes in $N_j(j^*)$ as follows:
- 8 $w_{ij}(t+1) = w_{ij}(t) + \eta_t[x_i^k - w_{ij}(t)]$, $j \in N_{j^*}$, $i = 1, 2, \dots, d$
- 9 until $\eta(t) < \eta_{\min}$
- 10 return $\mathbf{w}_j$, $j=1,2,\dots,c$
- 11 end

第九节 深度学习浪潮

6.1.3 人工神经网络发展历程



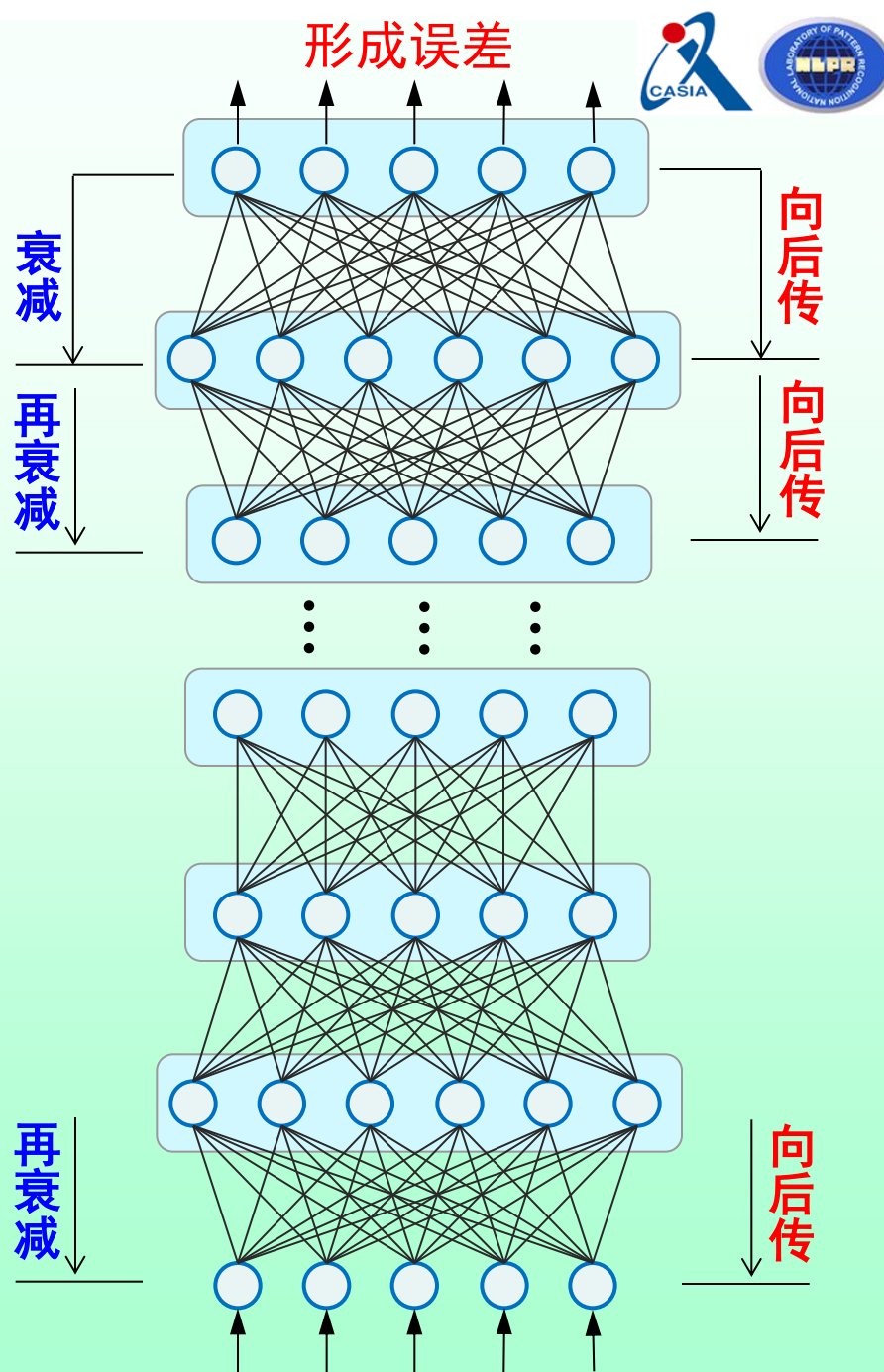
人工神经网络发展历程



人工神经网络发展历程

- 低谷中的主要挑战

- 网络结构单一
 - 参数过多
 - 可实现的任务单一
-
- 训练样本相对较少
 - 计算能力十分有限
 - 学习方法不够高效



6.9.1 深度学习浪潮！

Deep Learning Since 2006 (Sciences, vol. 313, pp. 504-507)

Reducing the Dimensionality of Data with Neural Networks

G. E. Hinton* and R. R. Salakhutdinov

两个重要的信息：

1. 具有多个隐层的人工神经网络具有优异的特征学习能力，学习得到的特征对数据有更本质的刻画，从而有利于可视化或分类；
2. 深度神经网络在训练上的难度，可以通过“逐层初始化” (Layer-wise Pre-training) 来有效克服。

Neural networks are coming back!

6.9.1 深度学习浪潮！

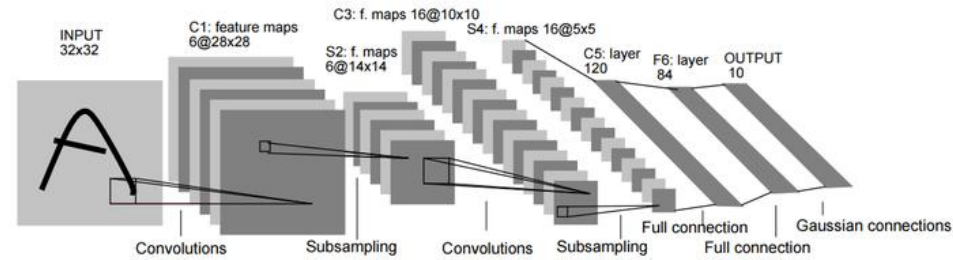
- 在语音识别取得重大突破



错误率降低
20–30%!

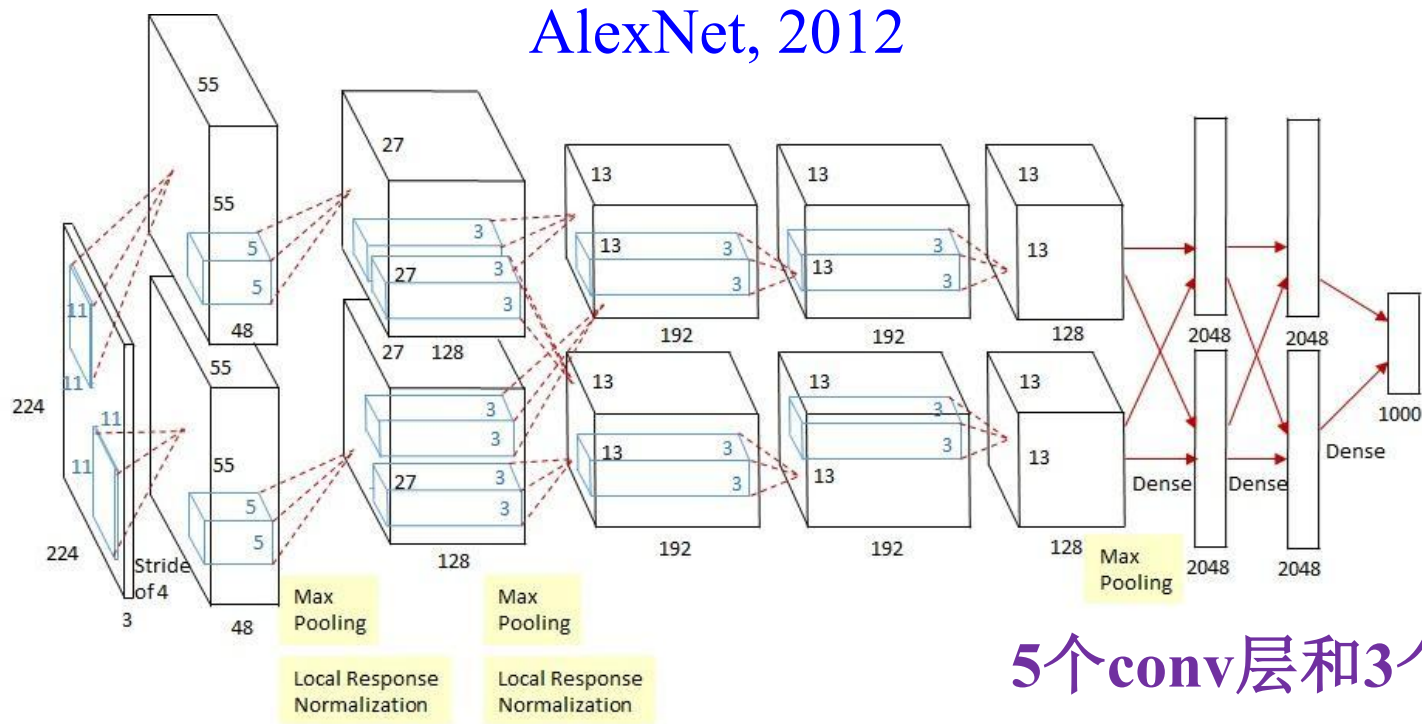
G. Hinton, etc. Deep Neural Networks for Acoustic Modeling in Speech Recognition. IEEE Signal Processing Magazine, 2012.

6.9.1 深度学习浪潮！



- 在图像分类取得突破

LeNet (1998): 2个conv层和2个FC层



5个conv层和3个FC层

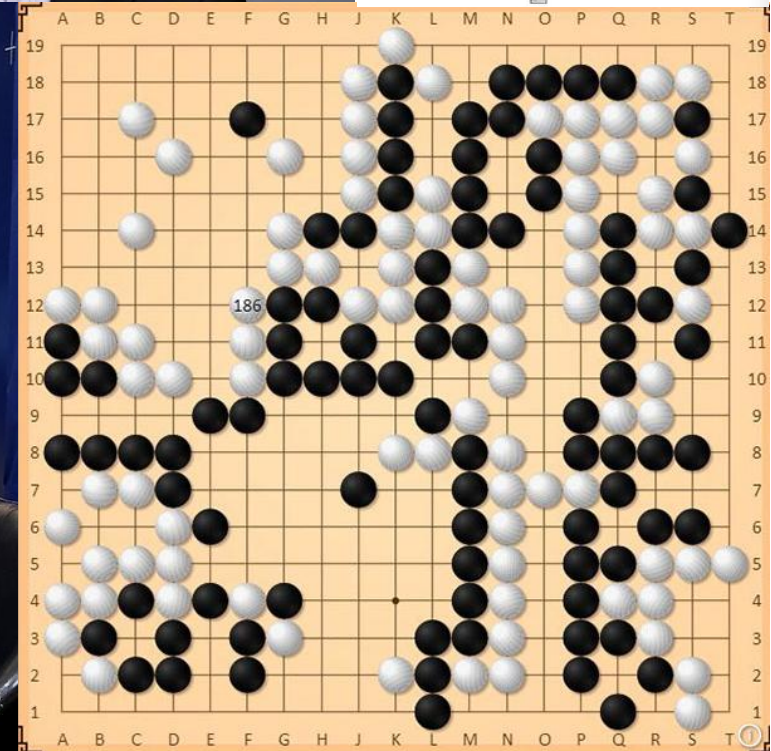
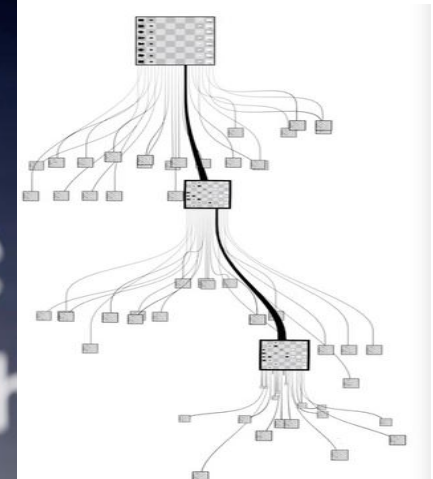
Alex Krizhevsky, Ilya Sutskever, Geoffrey E. Hinton. ImageNet Classification with Deep Convolutional Neural Networks. NIPS, 2012.

6.9.1 深度学习浪潮！

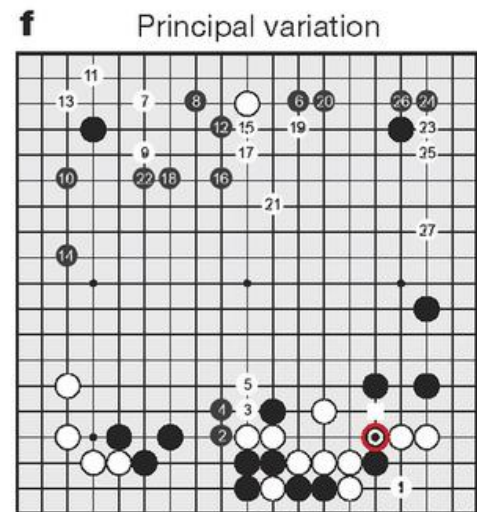
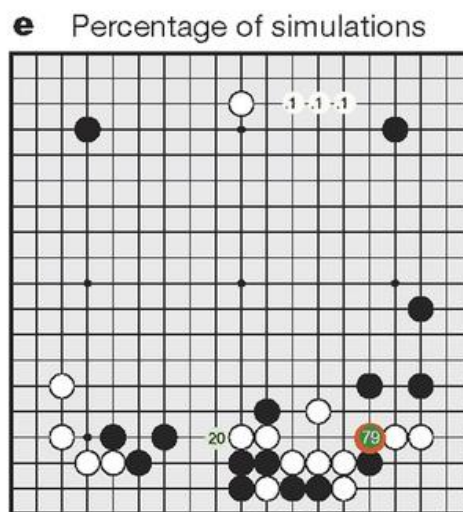
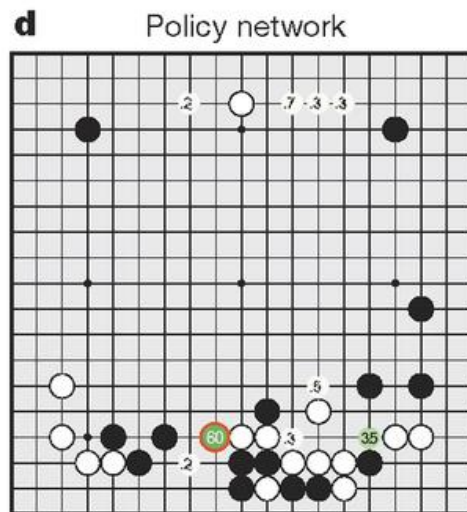
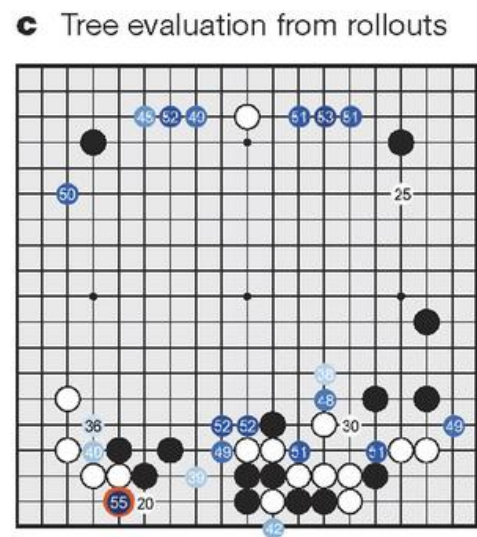
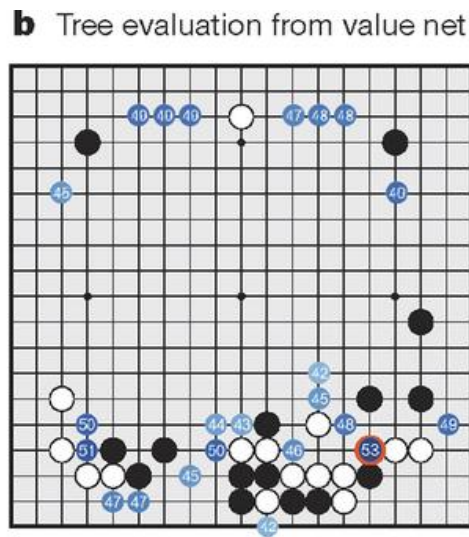
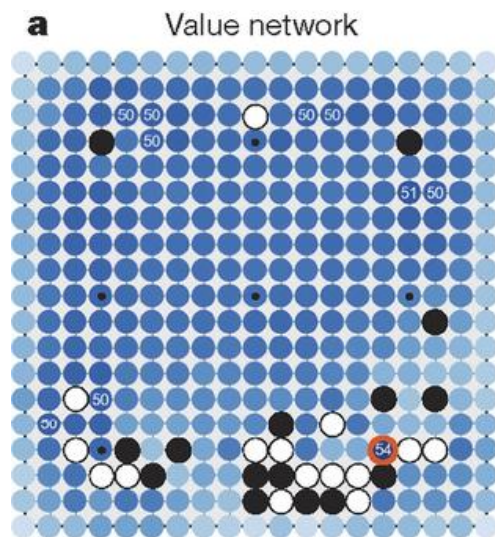
- Answer from G. Hinton, 2012.10



96%, 2015
93%, 2014
89%, 2013
85%, 2012
74%, 2011
72%, 2010



2016年3月9日，李世石在赛前和DeepMind的创始人D. Hassabis握手，AlphaGo最终以总比分4比1战胜李世石。



核心算法：强化学习+神经网络与蒙特卡洛搜索树

Silver D, et al. Mastering the game of Go with deep neural networks and tree search. Nature, 2016.
Silver D, et al. Mastering the game of go without human knowledge. Nature, 2017.

6.9.1 深度学习浪潮！

- 深度学习遍地开花

- LLM/MLLM: ChatGPT, Gemini, LLaMA, Qwen, DeepSeek
- 游戏: AlphaGo, AlphaStar
- 生物: AlphaFold
- 数学: AlphaGeometry, AlphaProof, GPT o1
- 地理、气象: GraphCast, 盘古大模型
- 编程: AlphaCode
- 自动驾驶: Tesla Autopilot/FSD

- AI的风险开始显现

6.9.1 深度学习浪潮！

- Academicals



Geoffrey Hinton



Jürgen
Schmidhuber



Yann Lecun



Yoshuo Bengio



Stephane Mallat



Andrew Ng



Li Fei-Fei



Honglak Lee



Ruslan
Salahutdinov



Kaiming He



Bruno A.
Olshausen



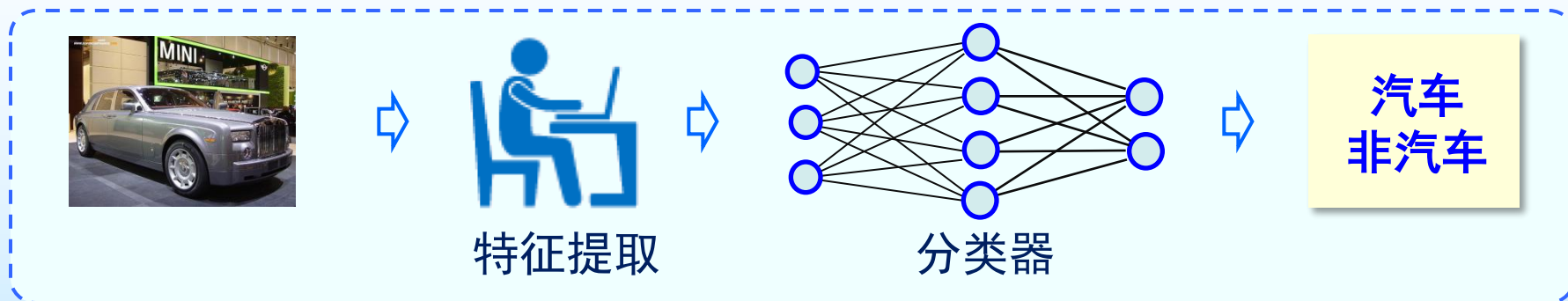
Rob Fergus

Most of them are focusing on Deep learning and Feature Learning.

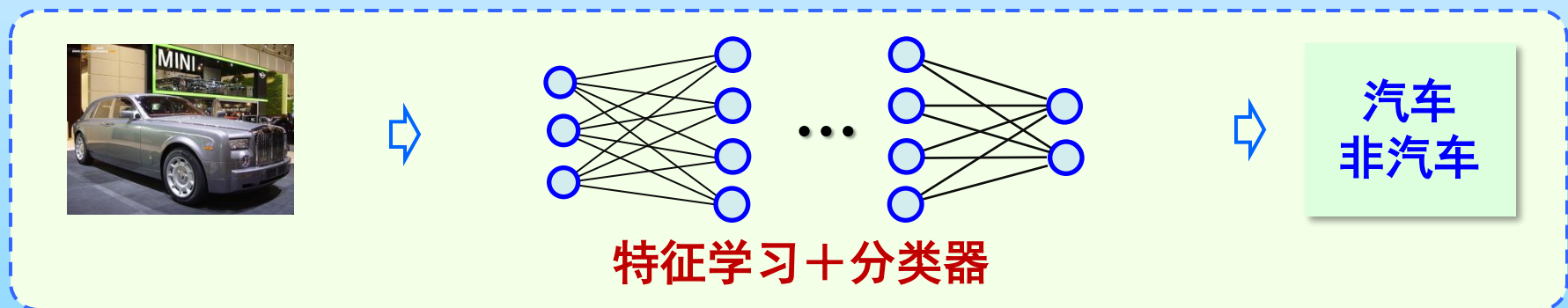
6.9.1 深度学习浪潮！

- Industrial
 - Microsoft & OpenAI: ChatGPT
 - Google & DeepMind: Gemini
 - Meta: LLama
 - Nvidia
 - Alibaba: Qwen
 - ByteDance: Doubao
 - DeepSeek: DeepSeek
 - Tencent

ML vs DL: 以分类为例



传统机器学习框架



深度学习框架

6.9.1 深度学习浪潮！

- 为什么有效？

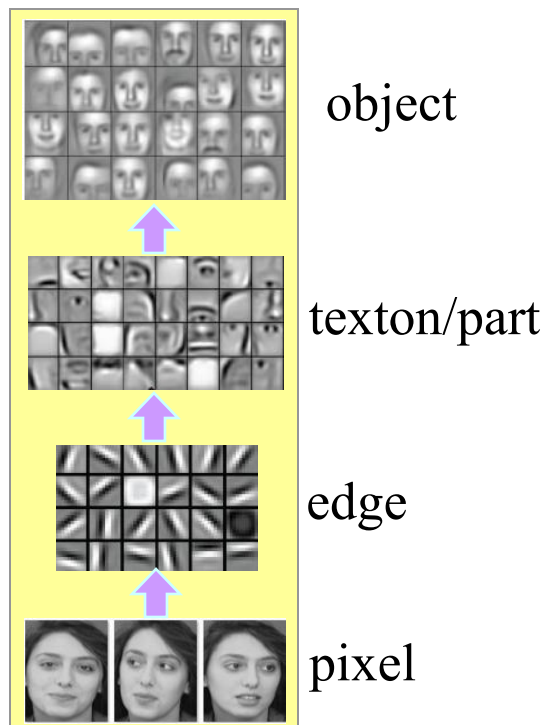
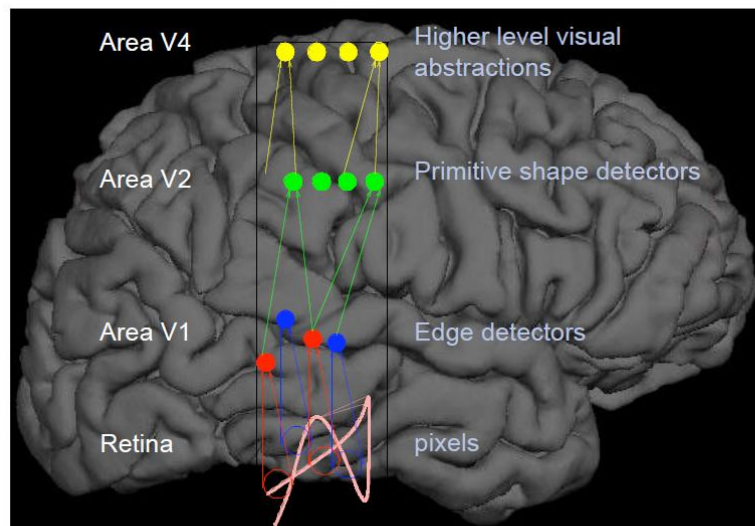
- 浅层神经网络可以近似任意函数，为何多层？

- 深层网络结构中，特别是卷积神经网络中，高层可以综合应用低层信息。
 - 低层关注“局部”，高层关注“全局”、更具有语义化信息。
 - 为自适应地学习非线性处理过程提供了一种可能的简洁、普适的结构模型。
 - 特征提取与分类器可以一起学习。

Hint from human learning

Deep Architecture in the Brain

为什么有效



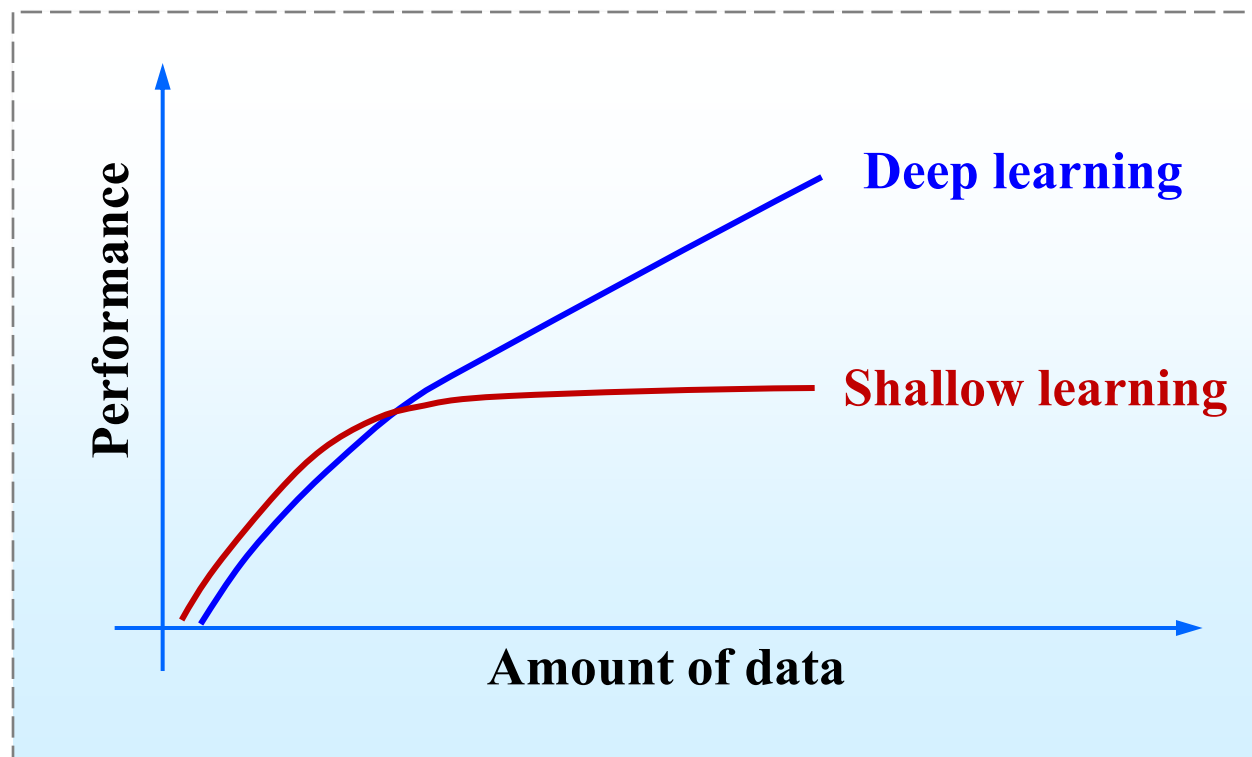
David Hubel和TorstenWiesel发现了视觉系统的信息处理方式，即**视皮层的分级特性**，获1981年诺贝尔生理学或医学奖。



Yoshua Bengio. Learning Deep architectures for AI. Foundations and Trends in Machine Learning, 2009

6.9.1 深度学习浪潮！

- 为什么有效 (Why Deep Learning)



How data science techniques scale with amount of data

6.9.1 深度学习浪潮！

• 为什么有效 (Why Deep Learning)

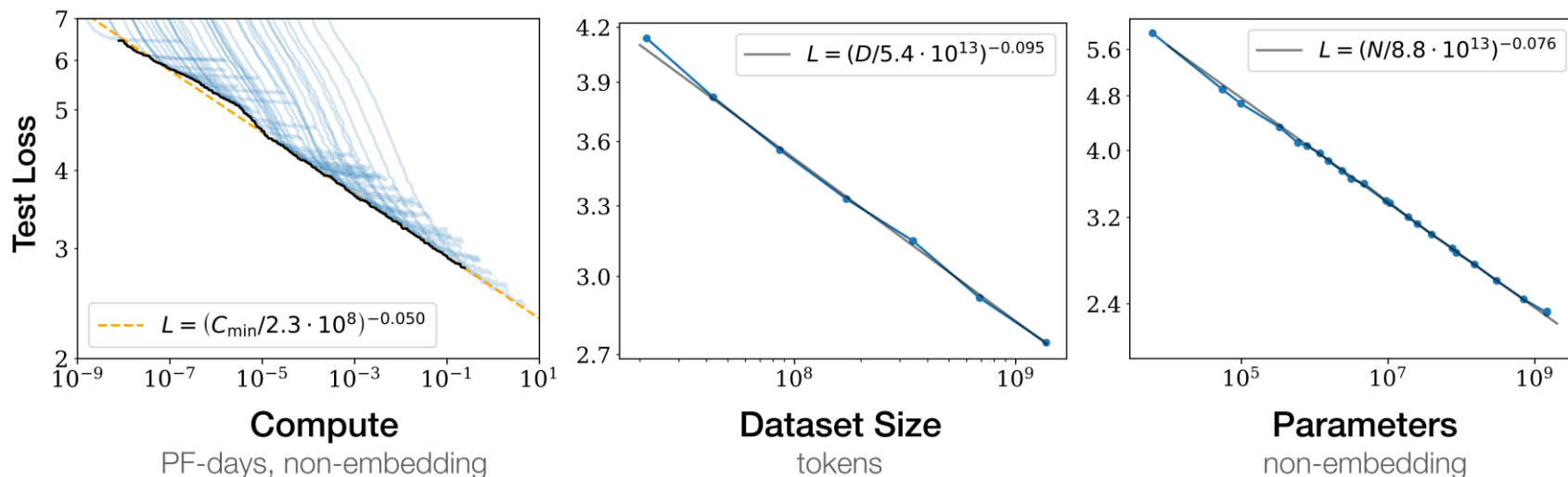


Figure 1 Language modeling performance improves smoothly as we increase the model size, dataset size, and amount of compute² used for training. For optimal performance all three factors must be scaled up in tandem. Empirical performance has a power-law relationship with each individual factor when not bottlenecked by the other two.

Scaling law: 模型的性能随训练数据量、模型参数量、训练所耗计算量的增加以幂律关系改善。



6.9.1 深度学习浪潮！

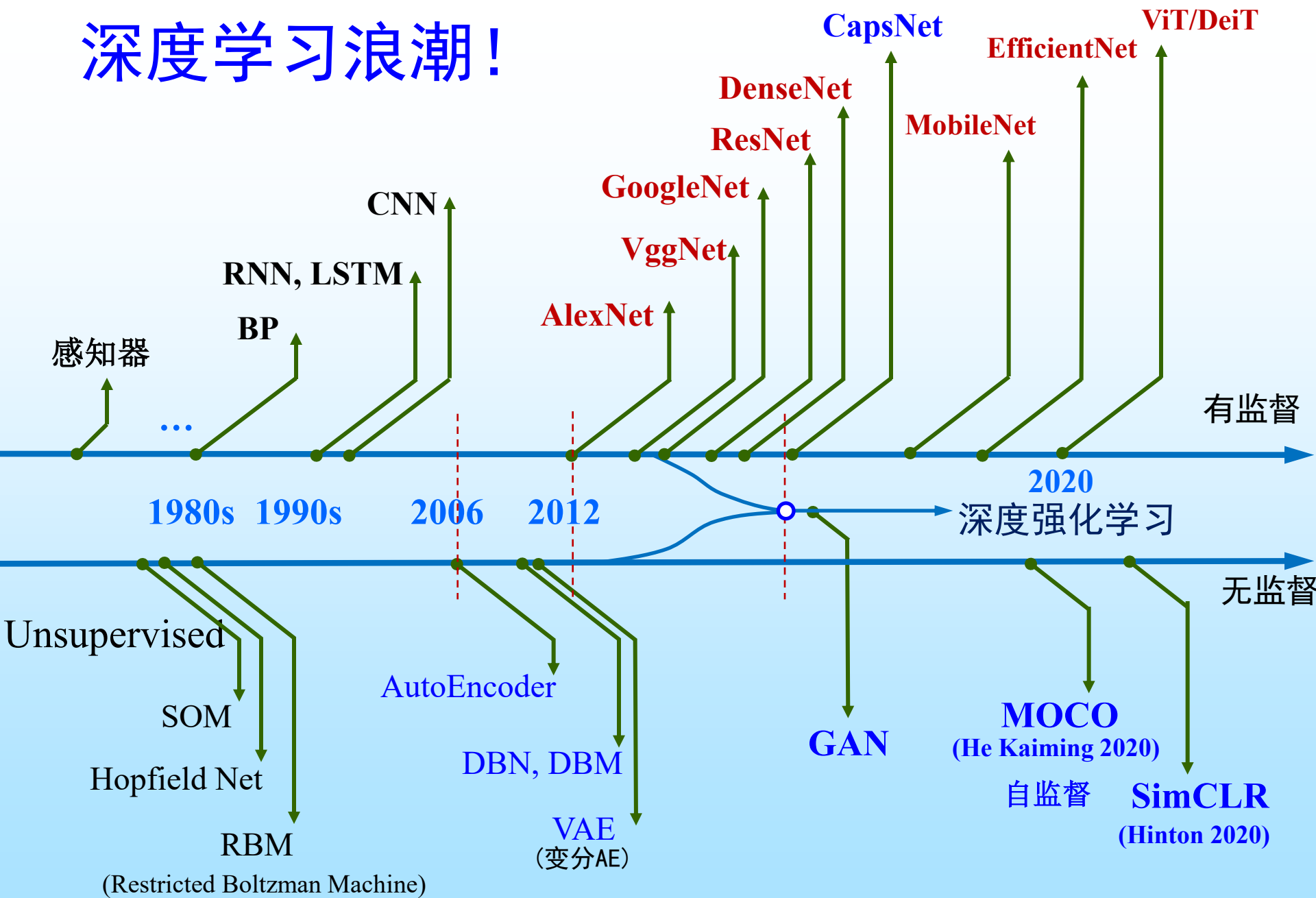
- 技术发展历程

- Multi-Layer Perceptron
- Hopfield network
- Boltzmann machine
- Restricted Boltzmann machine
- CNN
- RNN
- LSTM
- Autoencoder
- GNN
- Transformer

before 2000

Deep learning

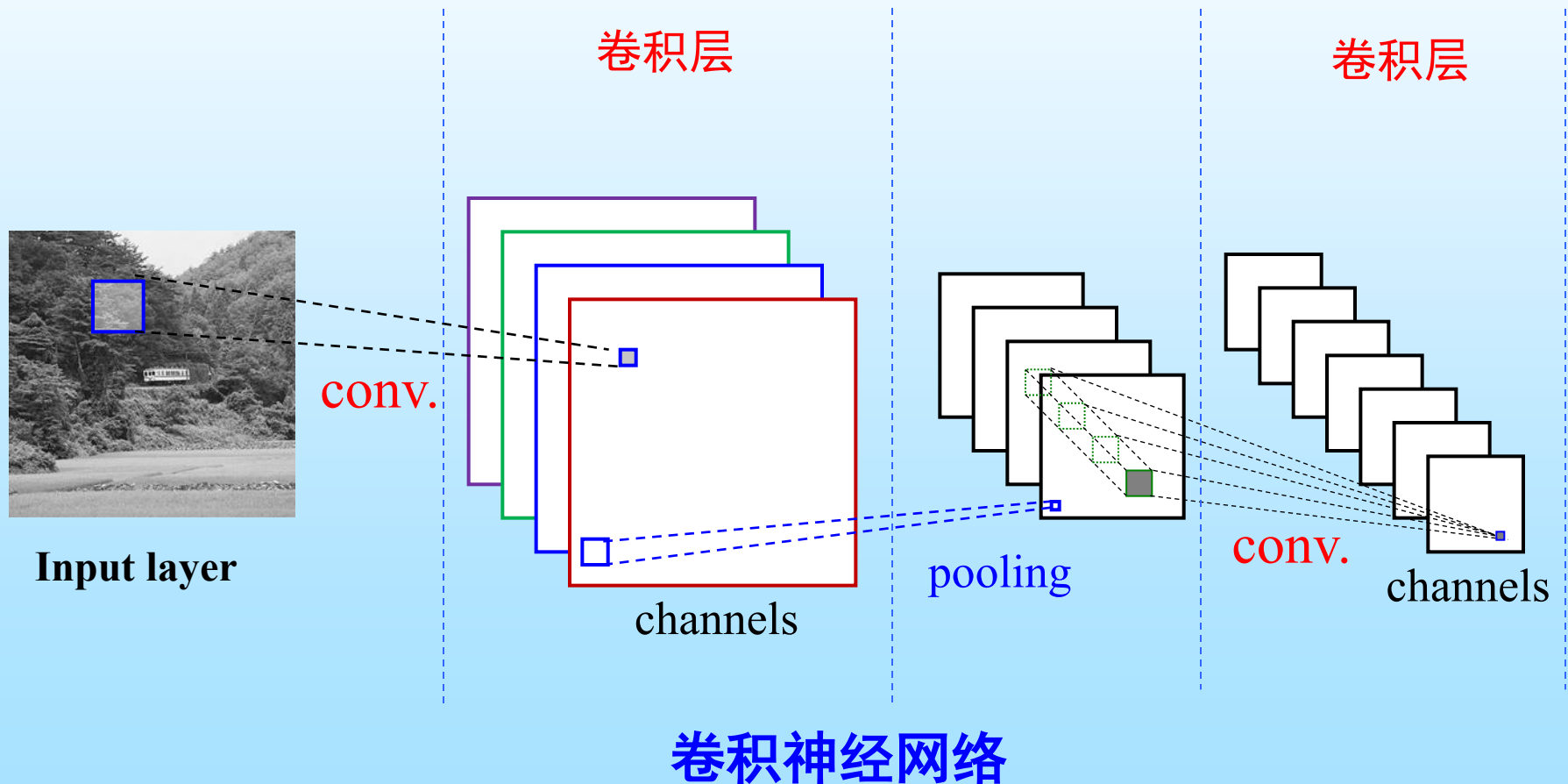
深度学习浪潮！



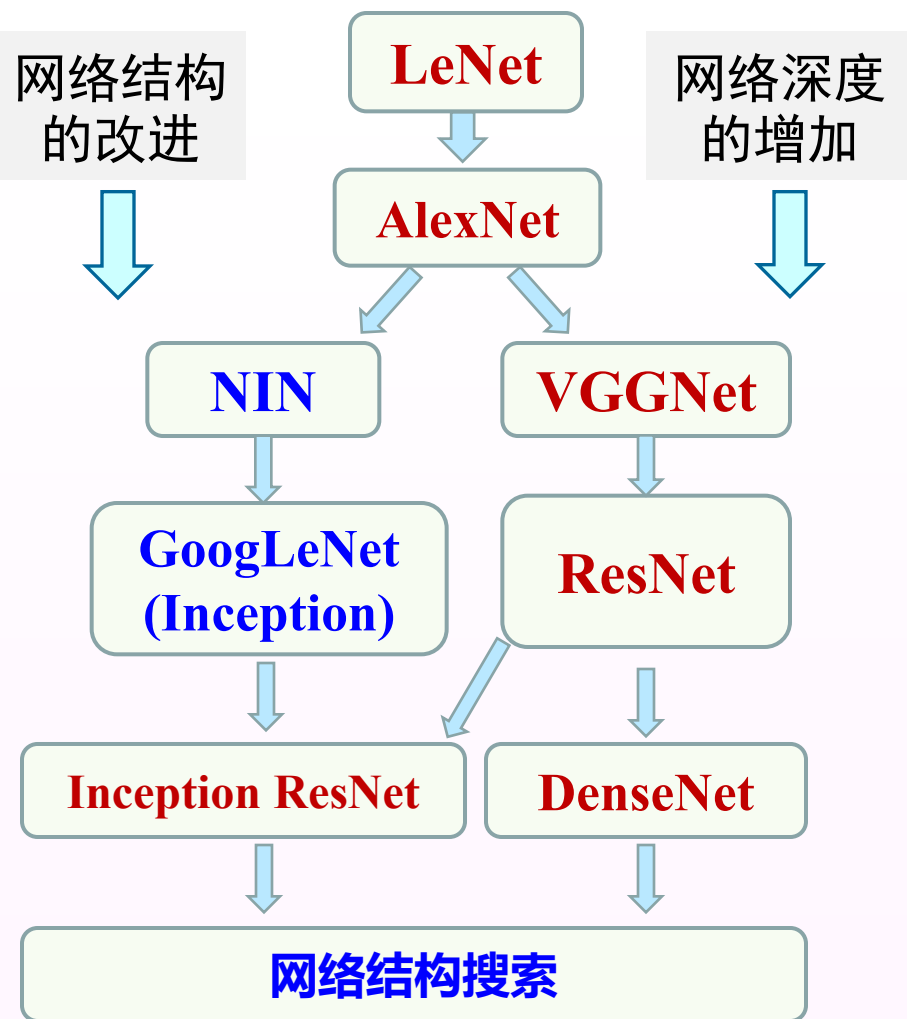
CV领域的深度学习技术发展之路

深度学习浪潮！——核心基础模型

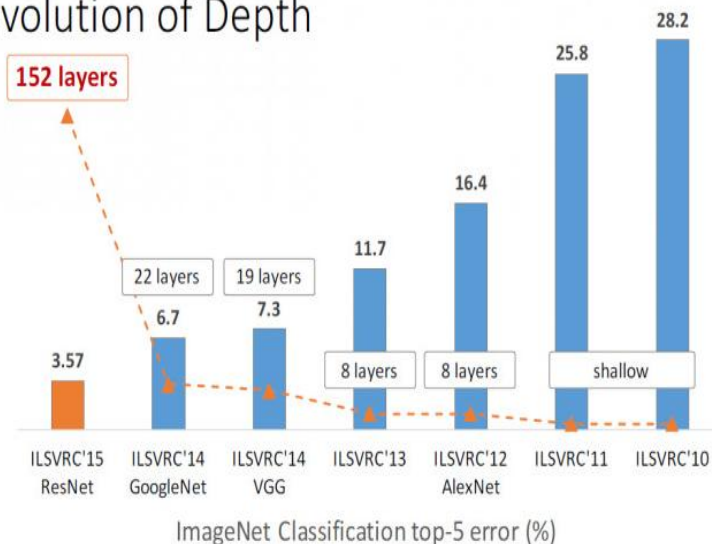
- ✓ 学习多个滤波器
- ✓ 每个滤波器对应一个所谓的通道 (channel)



深度卷积神经网络发展图

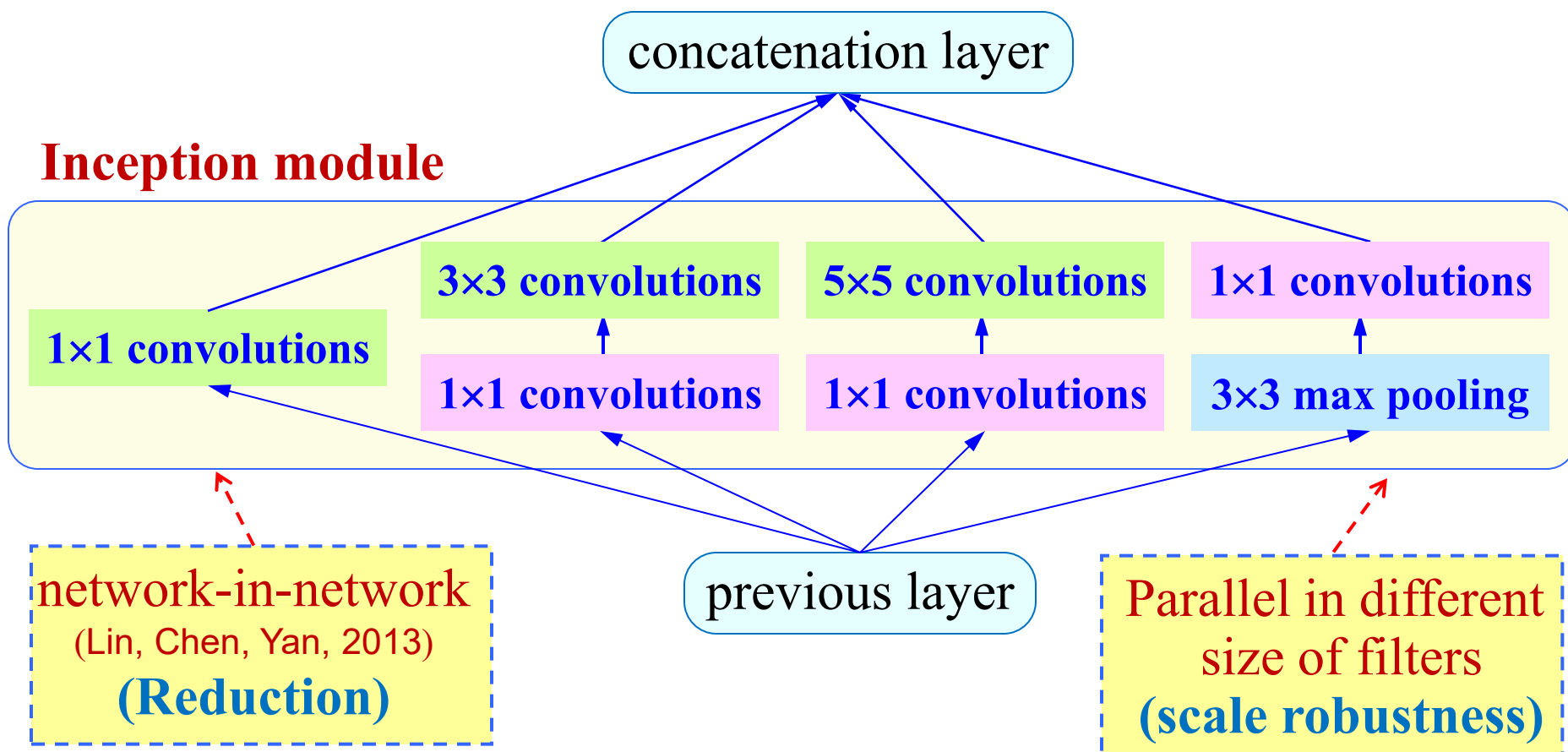


Revolution of Depth



ILSVRC图像分类竞赛结果

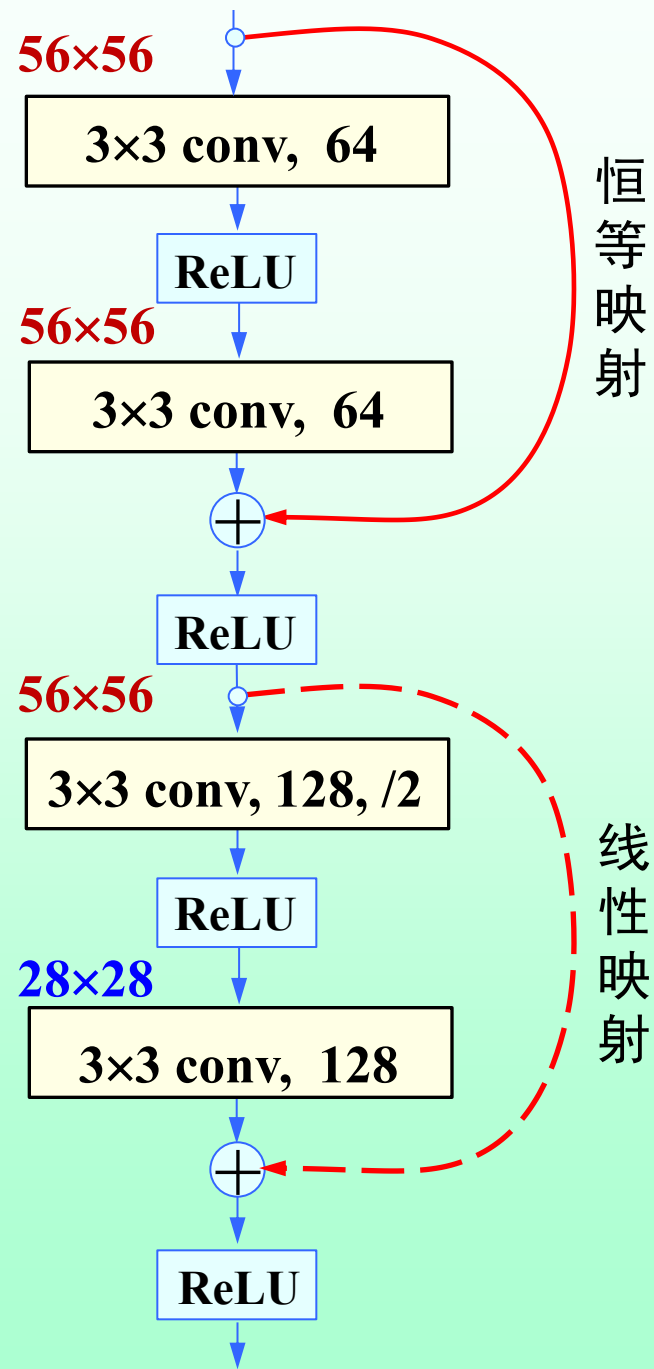
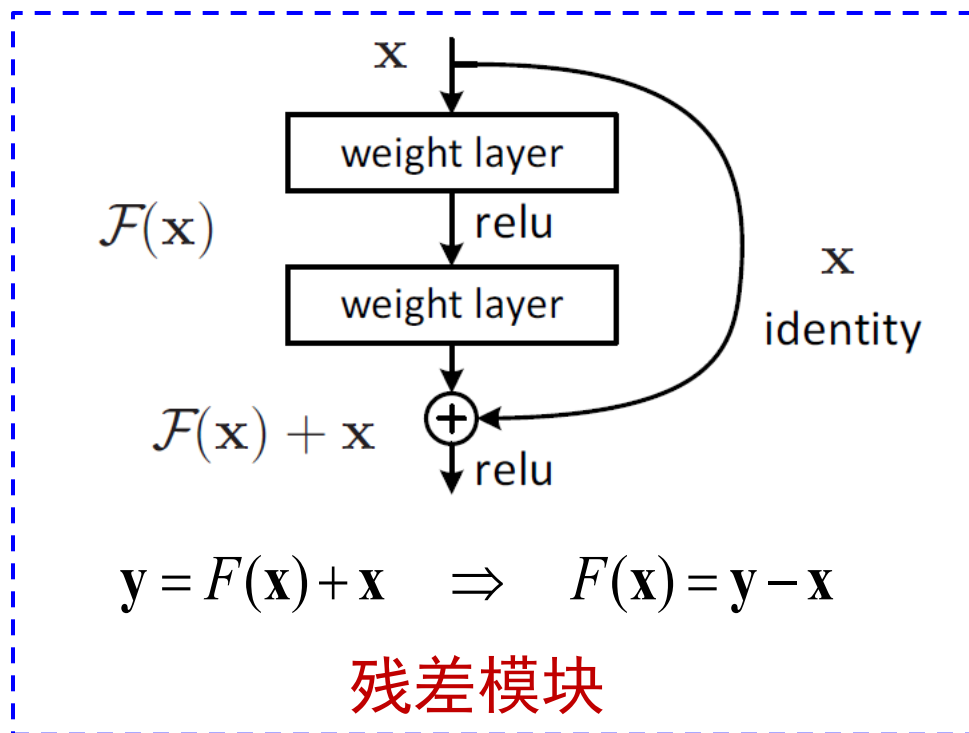
GoogLeNet: 增加网络宽度



- ✓ 多尺度多层次滤波:
- ✓ GoogLeNet的网络参数为AlexNet的1/12, ILSVRC 2014 top-5错误率降至6.67%。

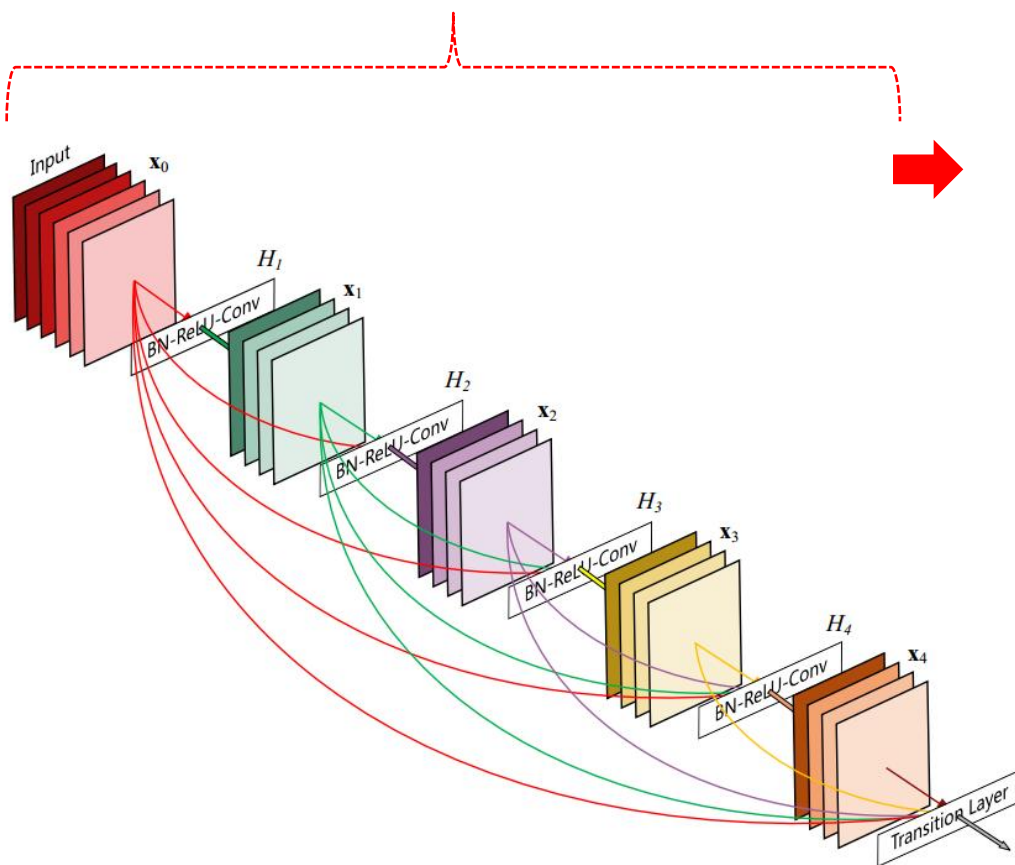
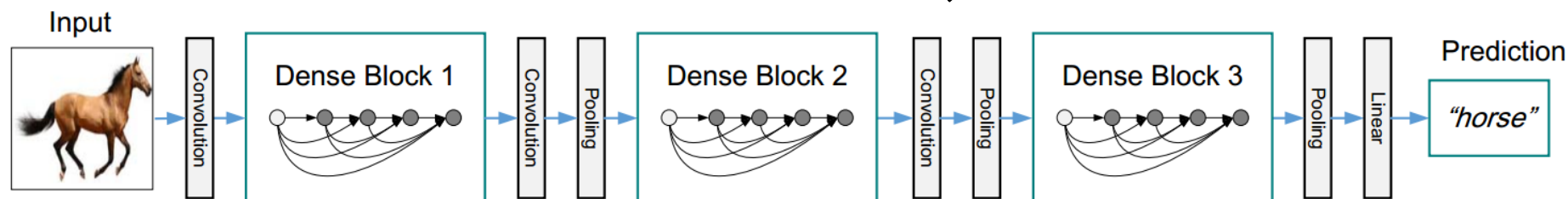
ResNet: 增加了信息传输的高速公路

- 输入通过shortcut与输出相加
- $F(\mathbf{x})$ 为残差映射，线性映射则用于维度匹配。



DenseNet: 密集连接

一个基于DenseNet的分类网络结构示意图，使用了3个dense block

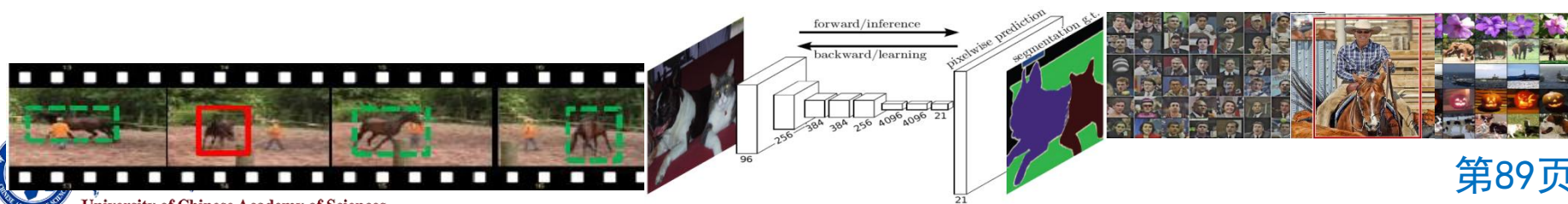


每个dense block有5层，每一层均直接与前面的所有层相连，实现特征的重复利用；同时把网络的每一层设计得特别“窄”，即只学习非常少的特征图(4个)，达到减少参数的作用。

基于DenseNet的分类器只需要ResNet一半的参数量，就可在ImageNet上达到相同分类精度

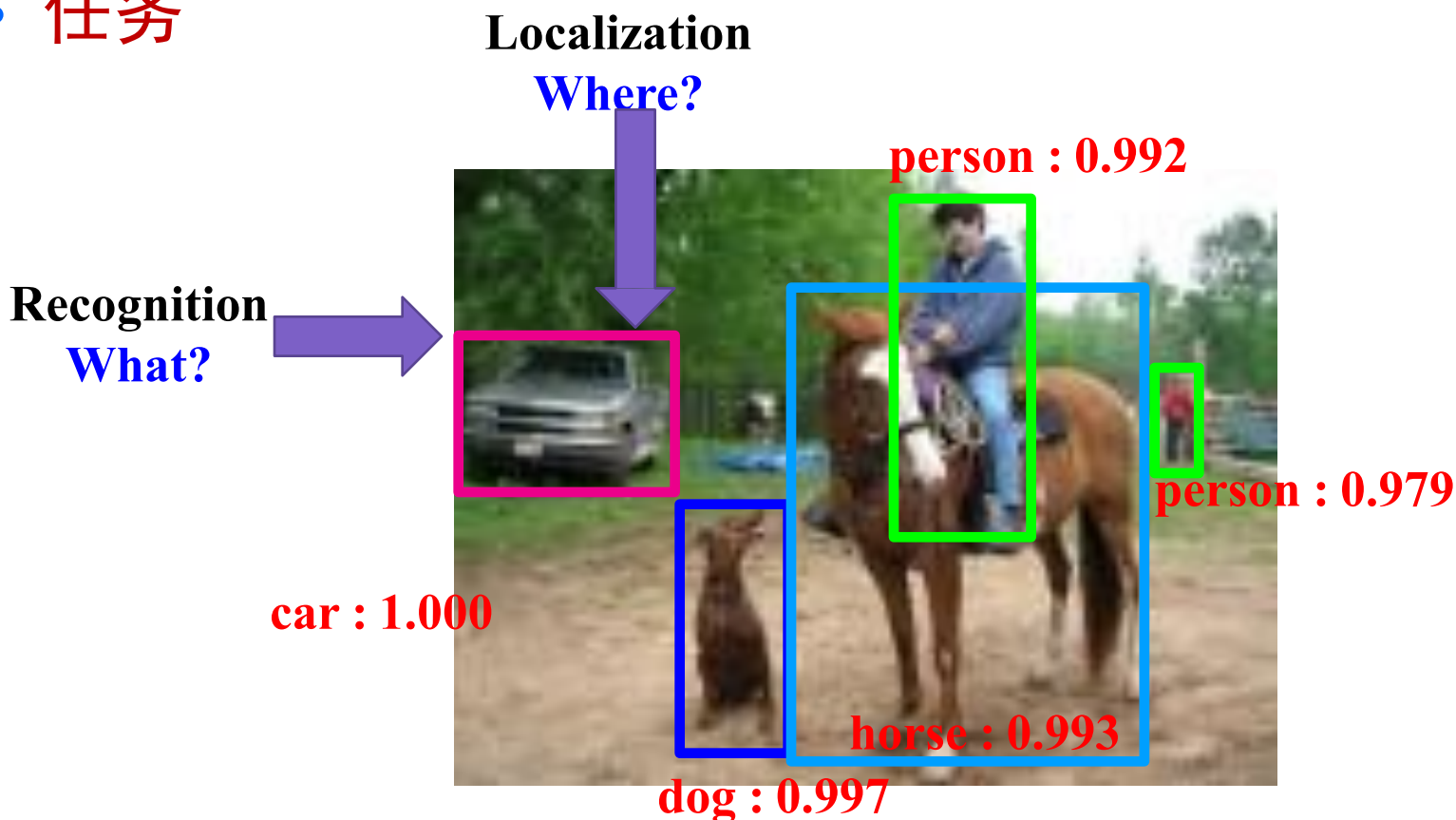
深度学习：在计算机视觉中的应用

- 随后在计算机视觉领域中的应用全面展开！
 - 图像分割、目标检测、图像分类
 - 目标跟踪
 - 图像标注、图像检索
 - 图像描述
 - 生物特征识别：人脸识别、人脸检测、指纹识别、虹膜识别
 - 行人检测、行人再识别、动作分类、事件分类
 - 深度估计、立体匹配、场景解析
 - 其它图像处理



深度学习：视觉目标检测

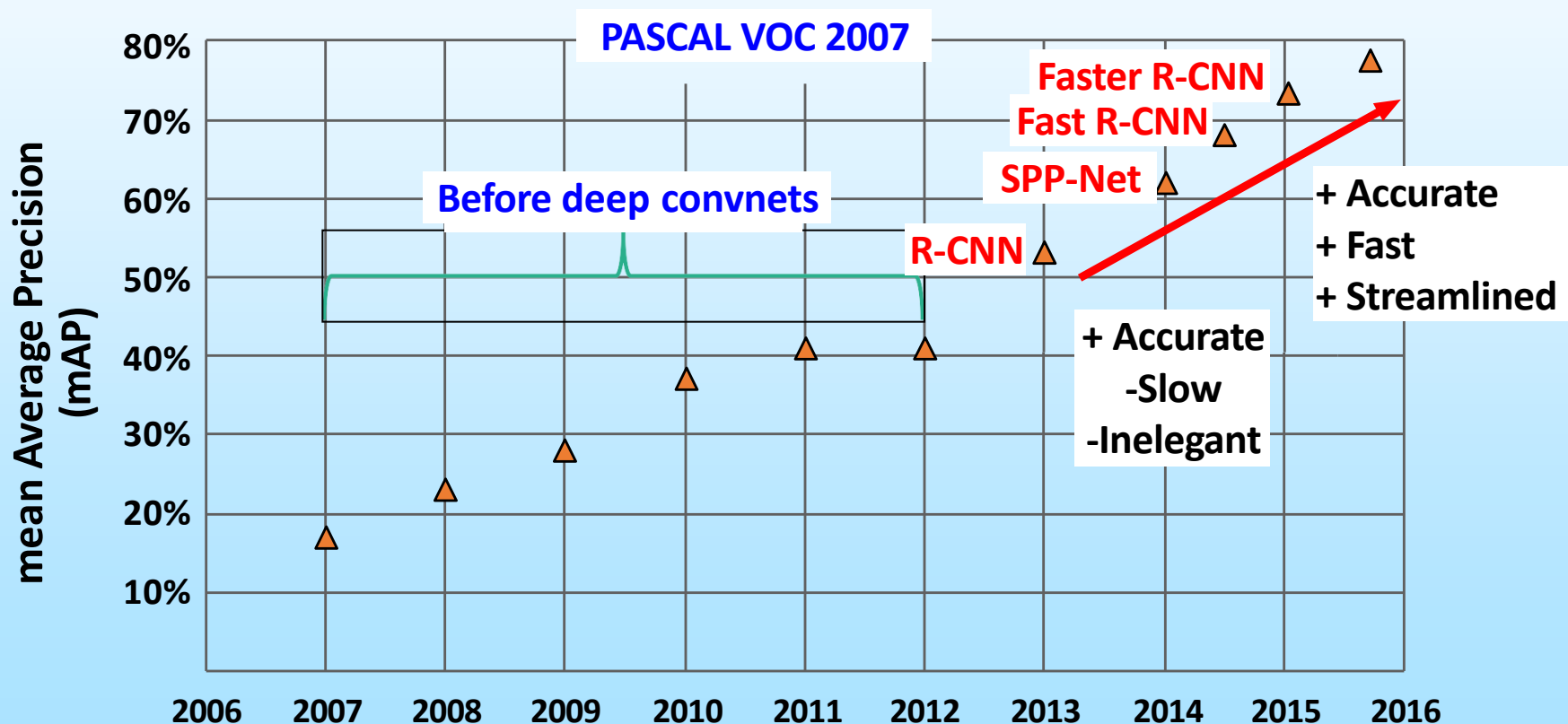
- 任务



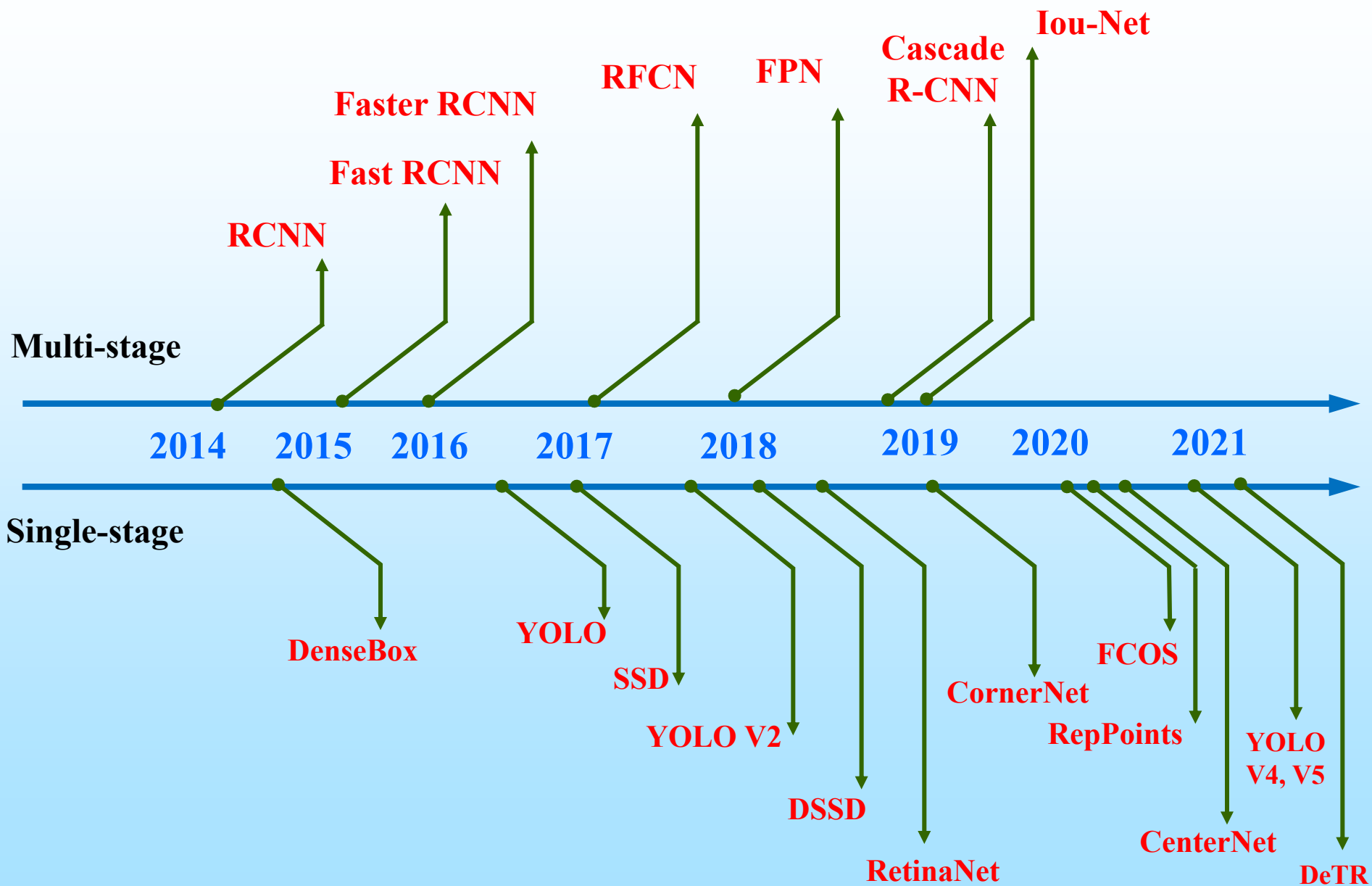
VOC2012: 20 classes. The train data has 11,530 images containing 27,450 ROI annotated objects

深度学习：视觉目标检测

- 目标检测：
 - R-CNN, SPP-Net, Fast R-CNN, Faster R-CNN**



视觉目标检测：深度模型发展之路



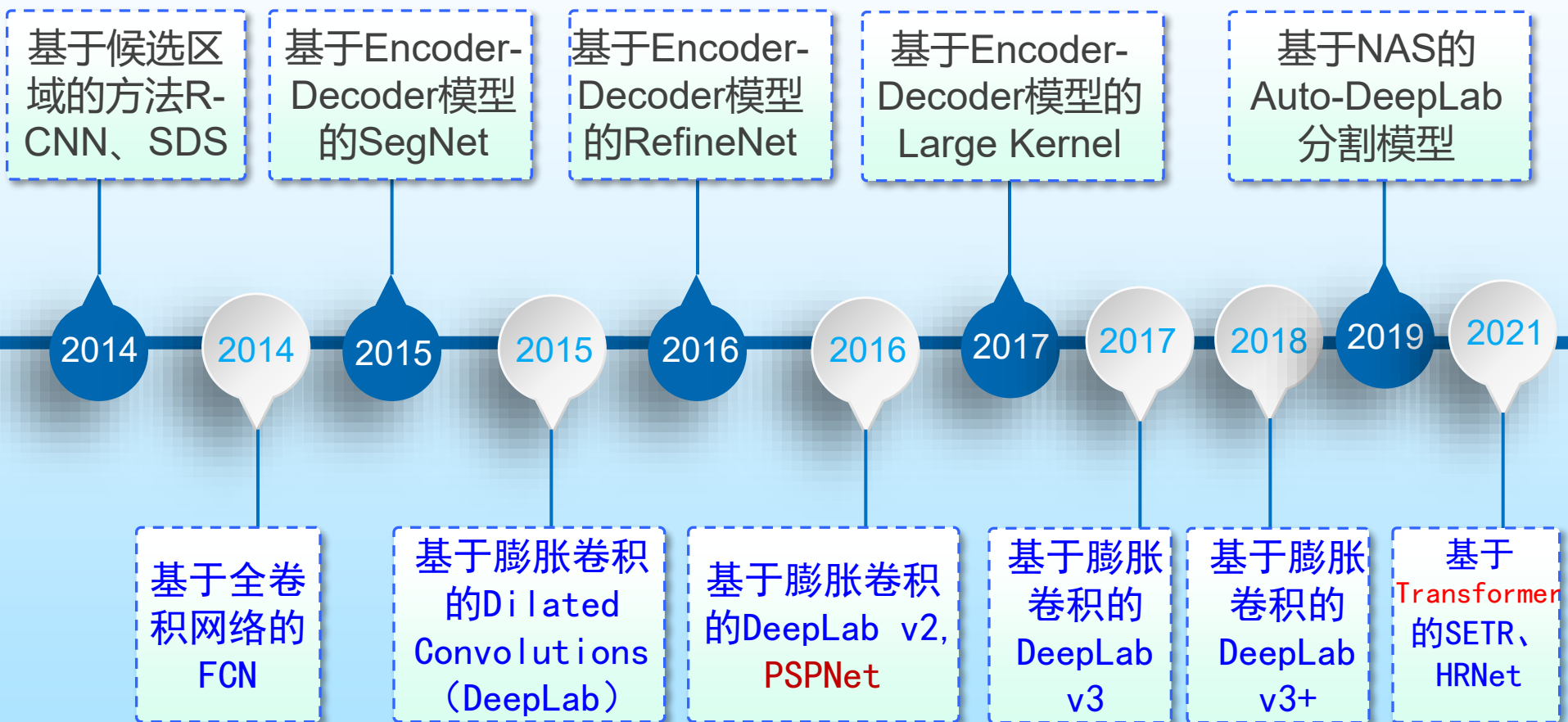
深度学习：图像语义分割

- Image segmentation is very challenging task in computer vision.



Segment image into different regions such that each region has the same semantic meaning.

图像分割：深层模型发展之路



深度学习：图像描述

- Image caption is a very-very challenging task in computer vision.



- ✓ A group of people of Asian descent watch a street performer in a wooded park area.
- ✓ A large crowd of people surround a colorfully dressed street entertainer.
- ✓ A crowd of people watching a balloon twister on a beautiful day.
- ✓ A crowd of people are gathered outside watching a performer.
- ✓ A crowd is gathered around a man watching a performance.

- ✓ O. Vinyals, A. Toshev, S. Bengio, and D. Erhan. Show and tell: A neural image caption generator. CVPR, 2015.
- ✓ K. Xu, J. L. Ba, R. Kiros, K. Cho, A. Courville, R. Salakhutdinov, R. Zemel, Y. Bengio. Show, Attend and Tell: Neural Image Caption Generation with Visual Attention, arXiv: 1502.03044v2, 2015.

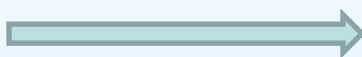
Image Caption: 技术之路



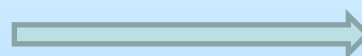
深度学习算法可迁移吗？



冬天到秋天



虚拟到现实

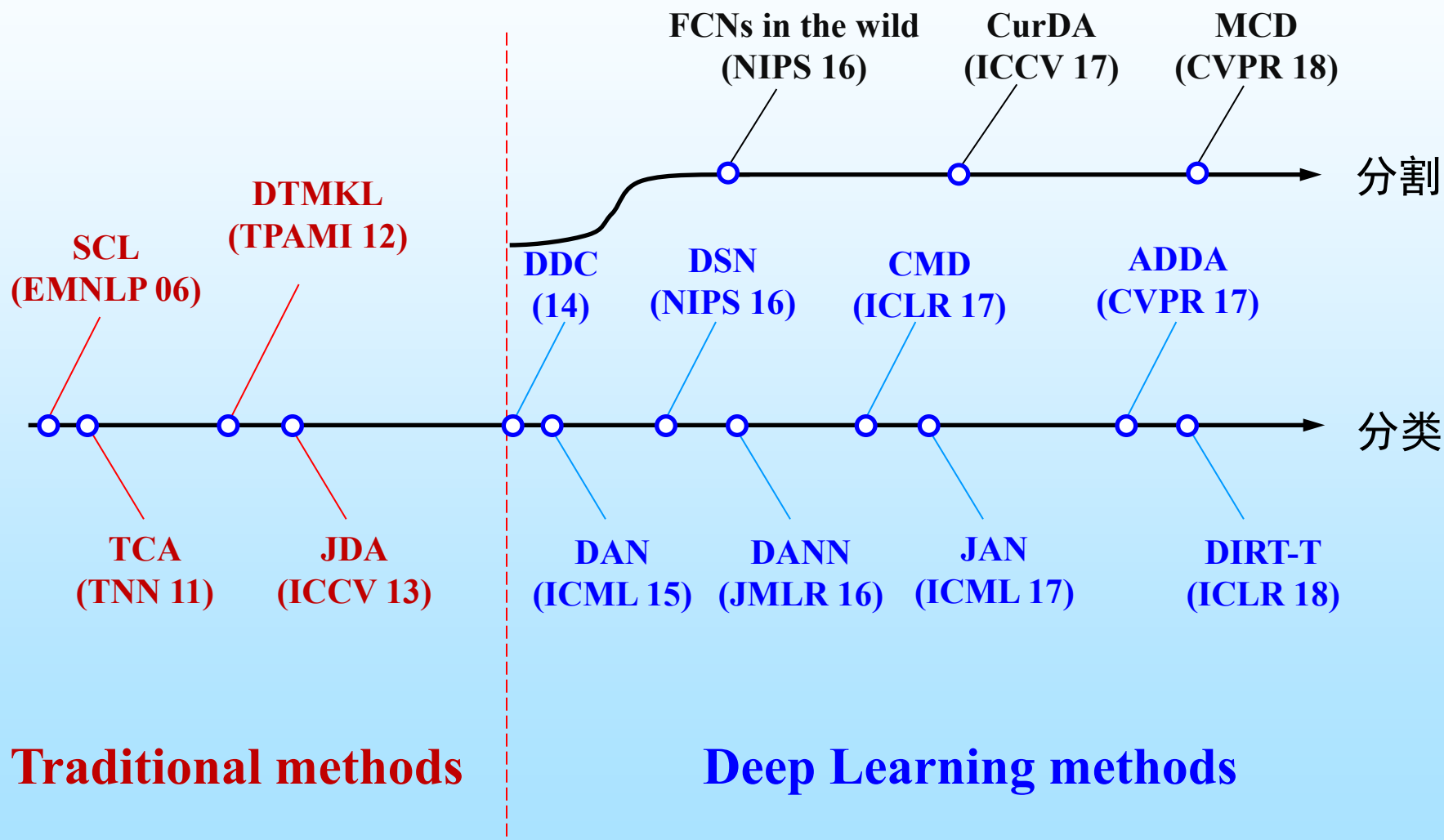


白天到黑夜



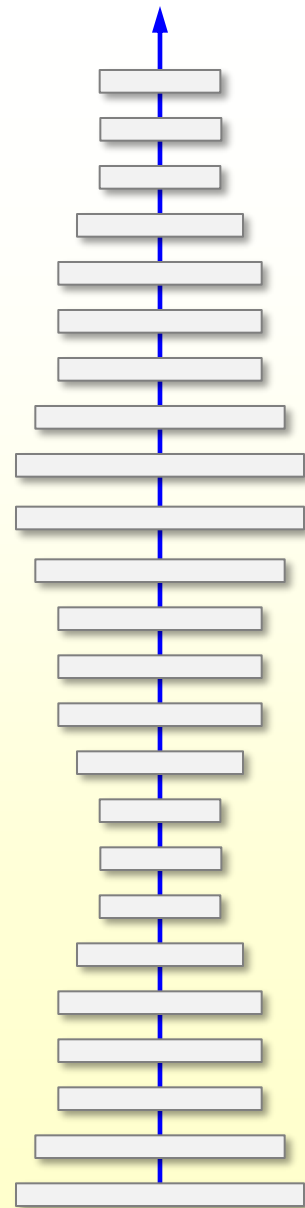
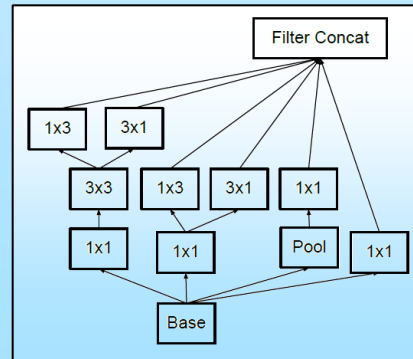
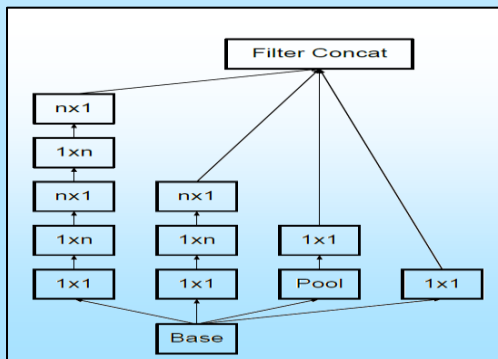
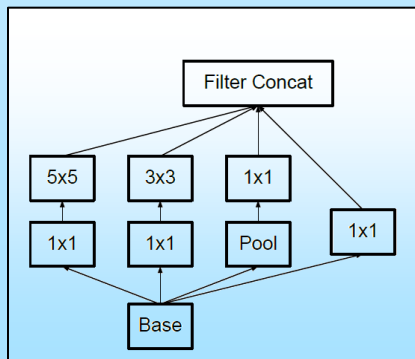
- ✓ 能让深度学习在不同的地点和季节之间进行时空迁移吗？
- ✓ 虚拟和现实之间能够迁移吗？

迁移学习域自适应研究历程

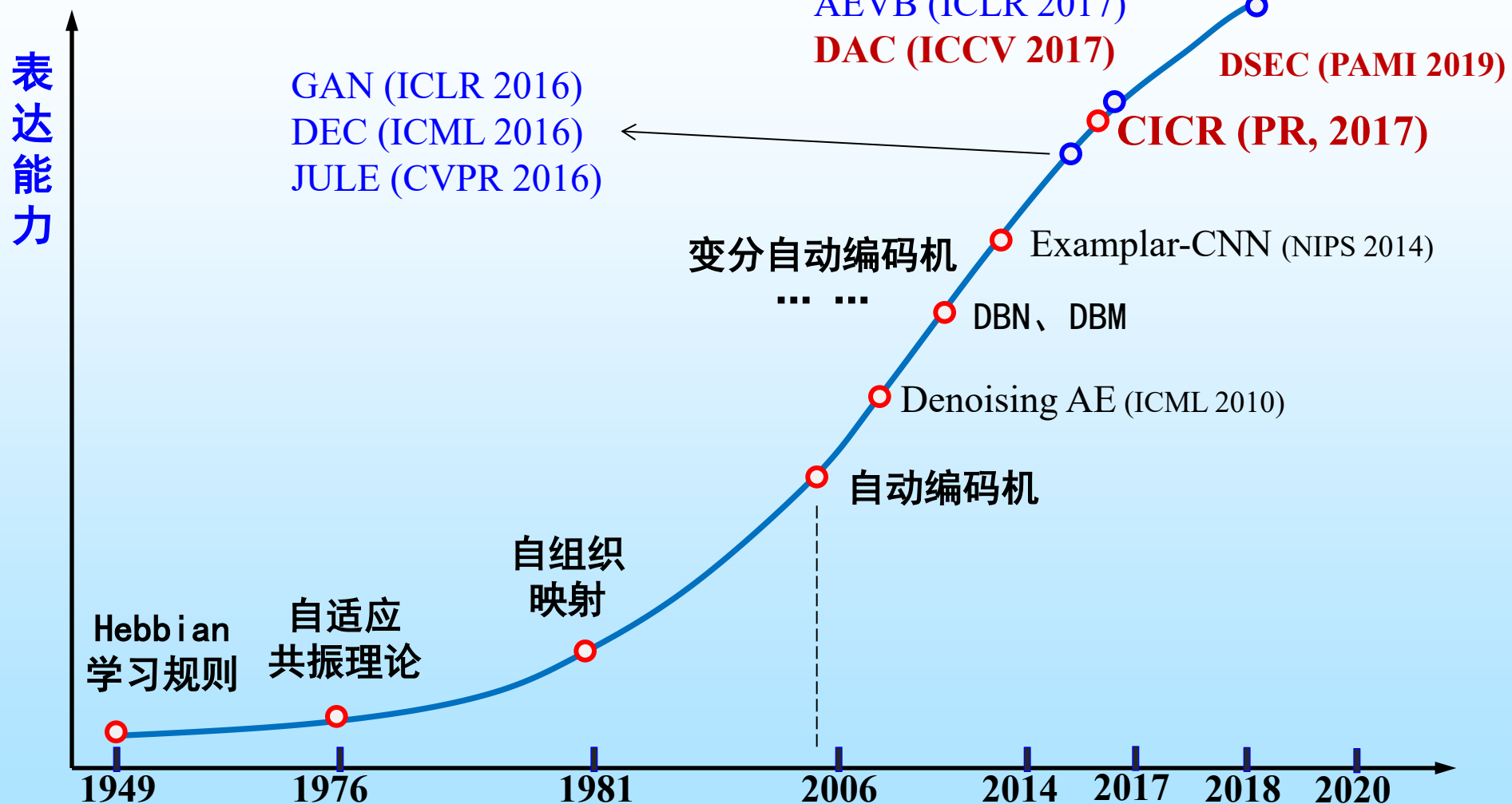


关于CNN的总体判断

- ✓ 以深度学习方法为核心的智能系统在**限定场景的既定感知任务中**取得了巨大进展。
- ✓ **积木式堆积**是当前网络结构设计普遍遵循的方法，并单纯地强调网络结构的“更深和更宽”，炼金术式的单任务性能追求。
- ✓ **固化的网络结构难以适应开放/变化环境自适应感知需求。**
- ✓ **网络结构的自主设计（AutoML），以NAS为代表！**



无监督学习

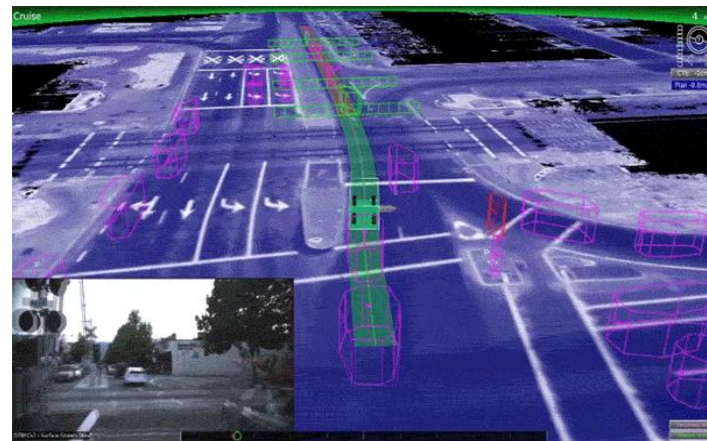


Chang, J., et al. Deep Adaptive Image Clustering. ICCV 2017, IEEE T-PAMI 2019

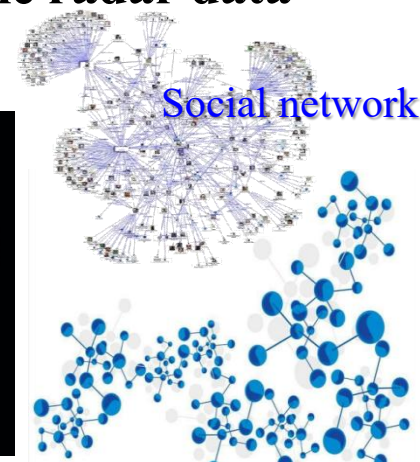
Chang, J., et al. Deep unsupervised learning with consistent inference of latent representations. *PR*. 2017

图卷积神经网络

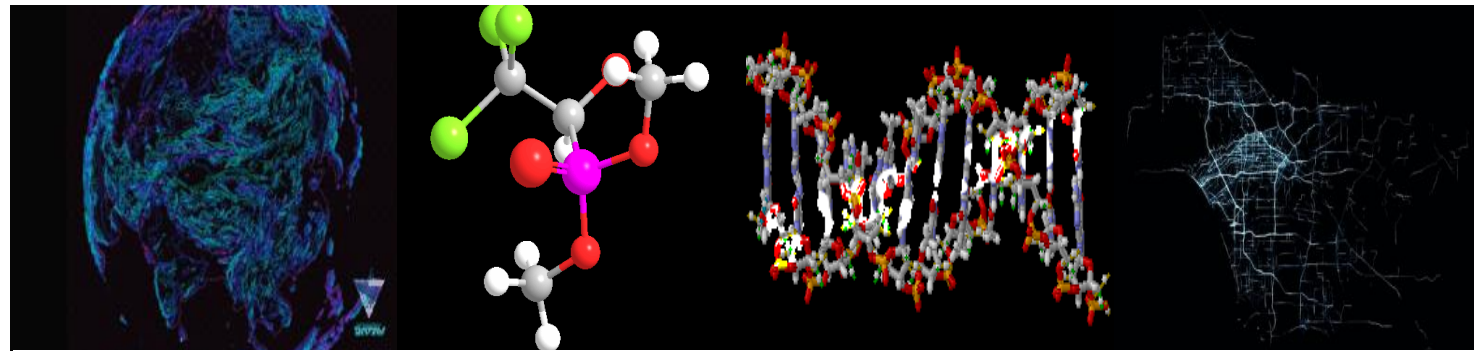
- ✓ **Data embedding on irregular grids**
 - **Social network:** Facebook, Twitter, Tencent.
 - **Infrastructure network:** transportation, Internet, electric power network, ...
 - **Biology/ecology/ Geography/ meteorology:** genes, molecules, brain connectivity, etc.



automatic radar data



molecules



3D data

molecules

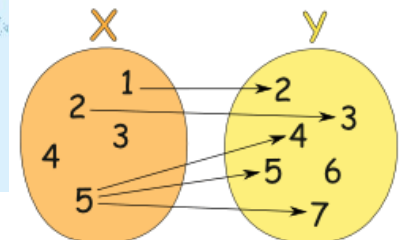
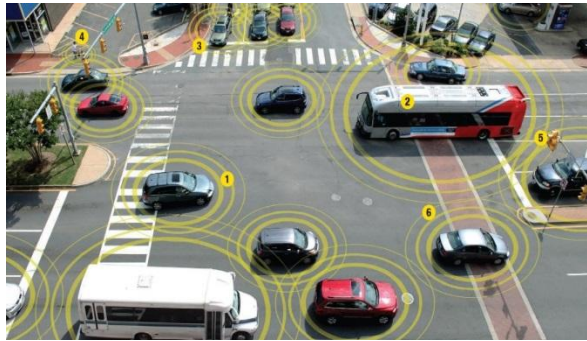
genes structure

transportation
network

图卷积神经网络

✓ Situations: **Data with relations**

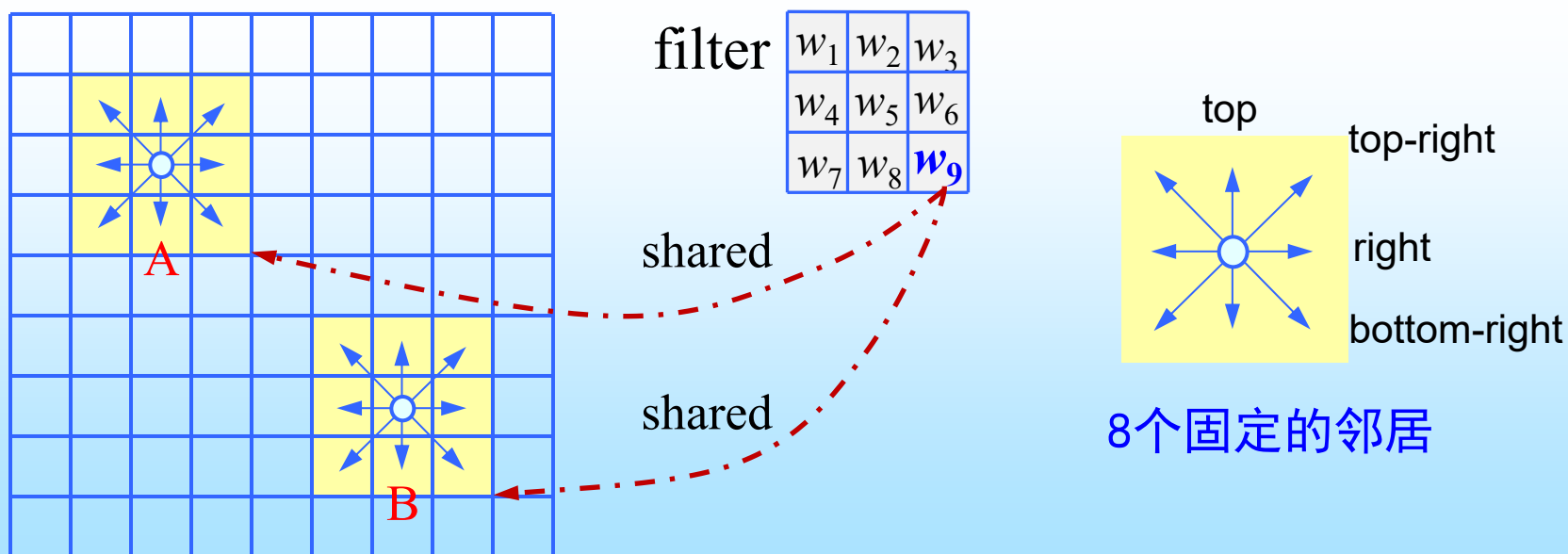
- Relations between **persons /circles /communities /groups /cities**
 - Recommendation system, ...
- Relations between **webpages** (page ranking)
- Relations between **cars in network of vehicle**
- Relations between **neurons**
- Relations between **functions in mathematics**



Data with non-gridded format

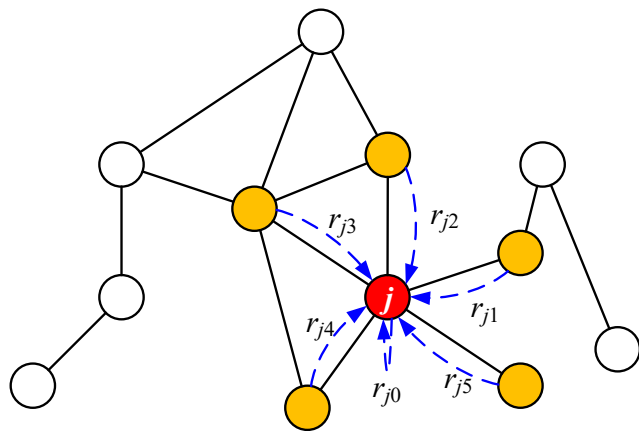
回忆图像卷积操作:

- ✓ For patch A and patch B, **they share the same filter** when performing the convolution operations.
- ✓ By **viewing the image as a graph** (each pixel as a node), **each node has the same neighbors**.

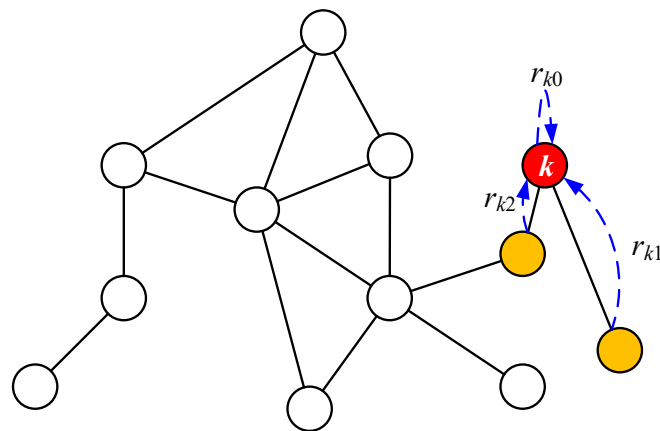


图卷积神经网络

- 我们的任务：
 - How to develop convolution operations for graph data?
 - **Just Graph Neural Network!**



j : 6 signals considered



k : 3 signals considered

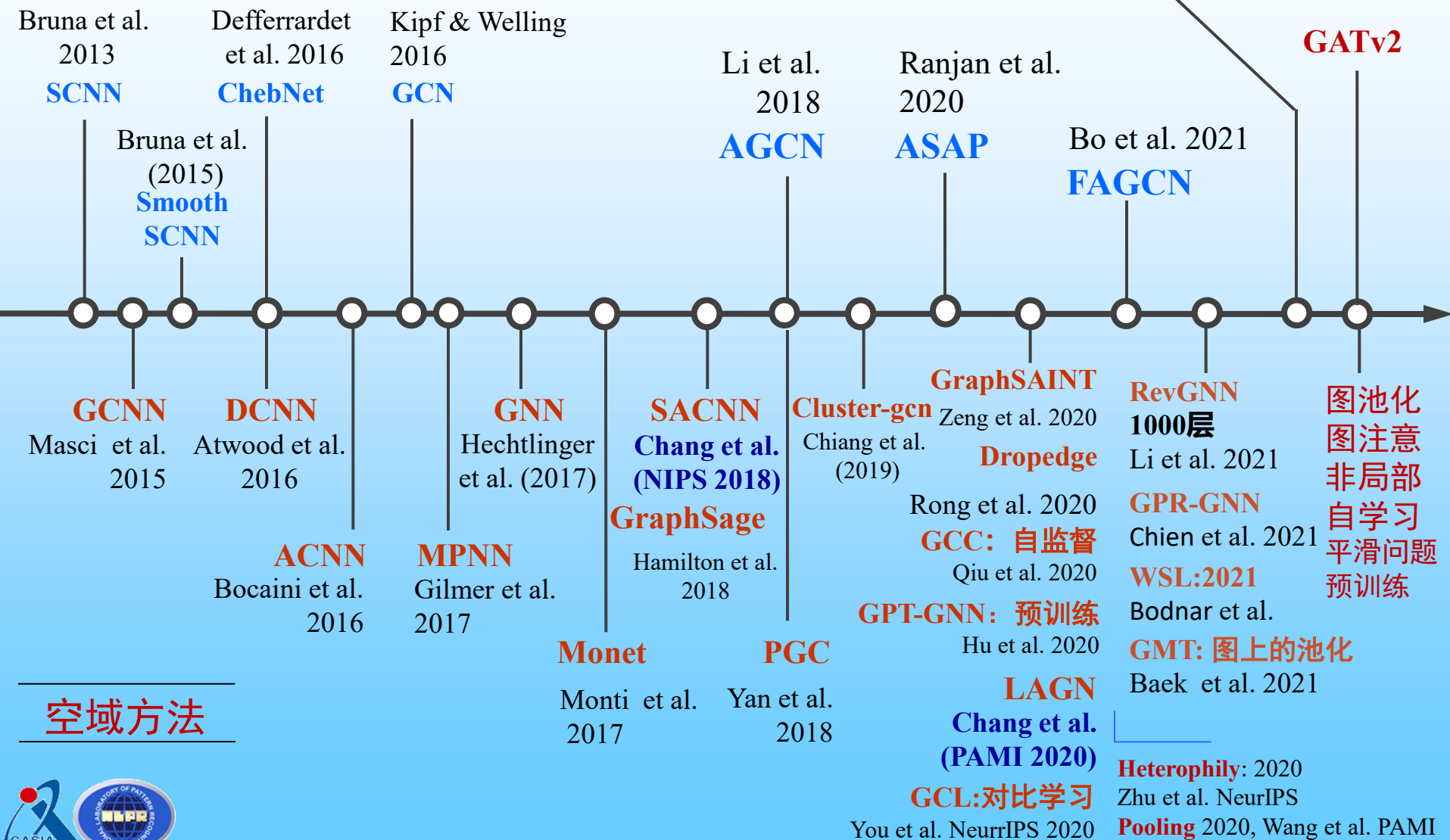
每个结点具有不同个数的邻近结点，且结点无序

图网络的主要研究方向

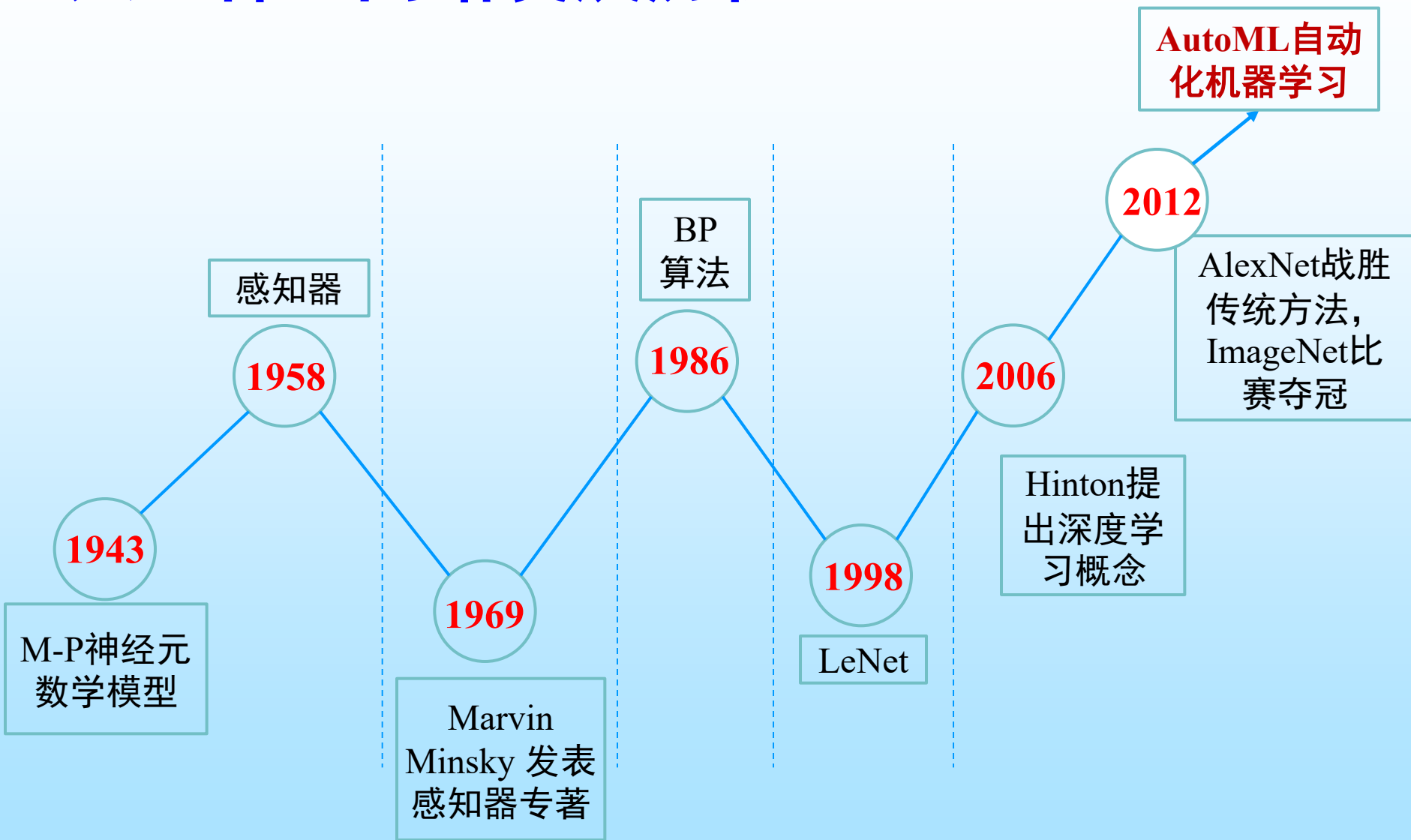
- **There are a few fundamental topics:**
 - **Graph Convolution Neural Networks (GCNN)**
 - Graph Attention Networks
 - Graph Auto-encoders
 - Graph Generative Networks
 - Graph Spatial-temporal Networks
 -
- **图卷积神经网络：深度学习中的热点研究方向！**

图卷积神经网络：研究之路

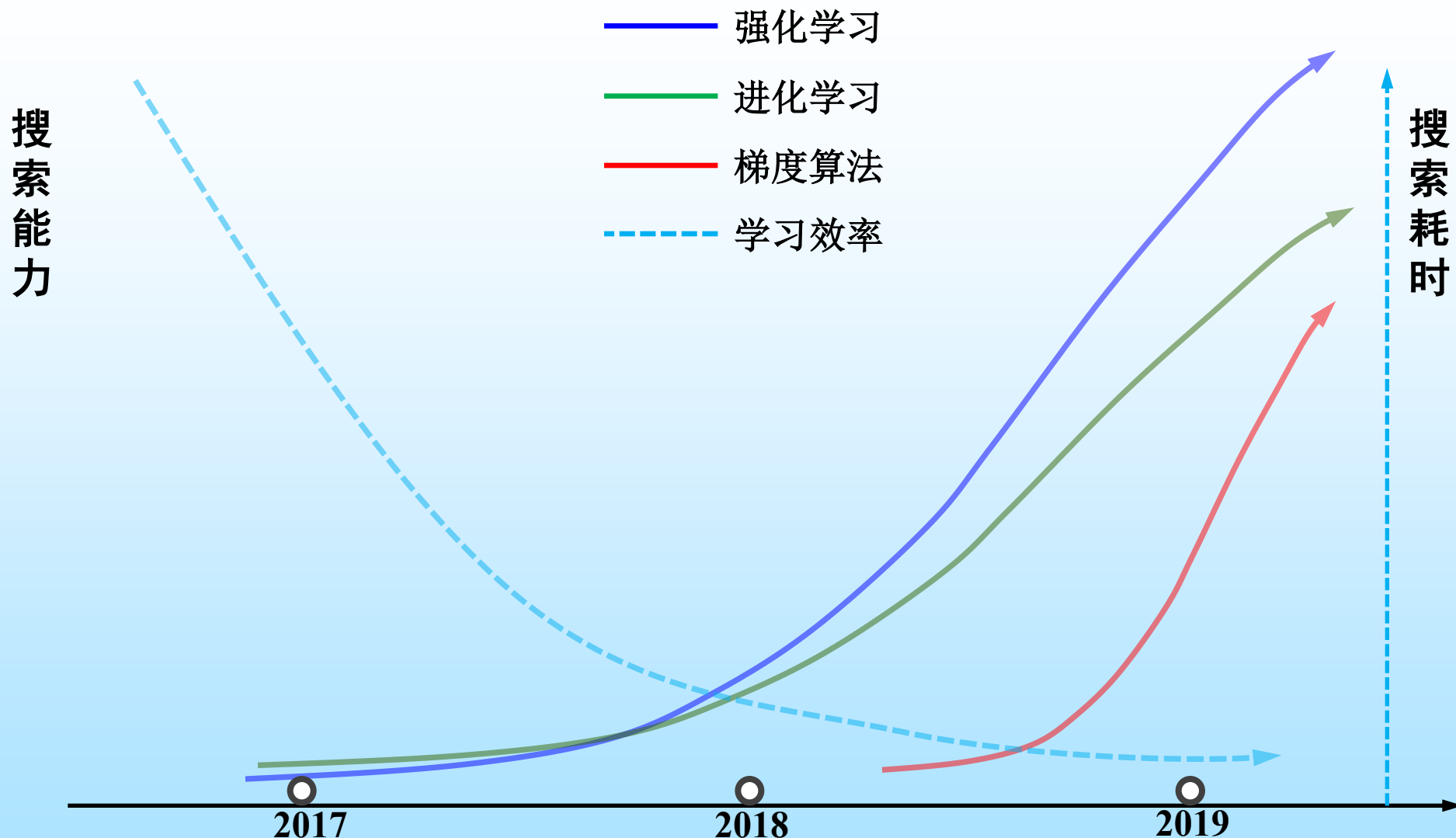
谱域方法



人工神经网络发展历程



网络结构搜索

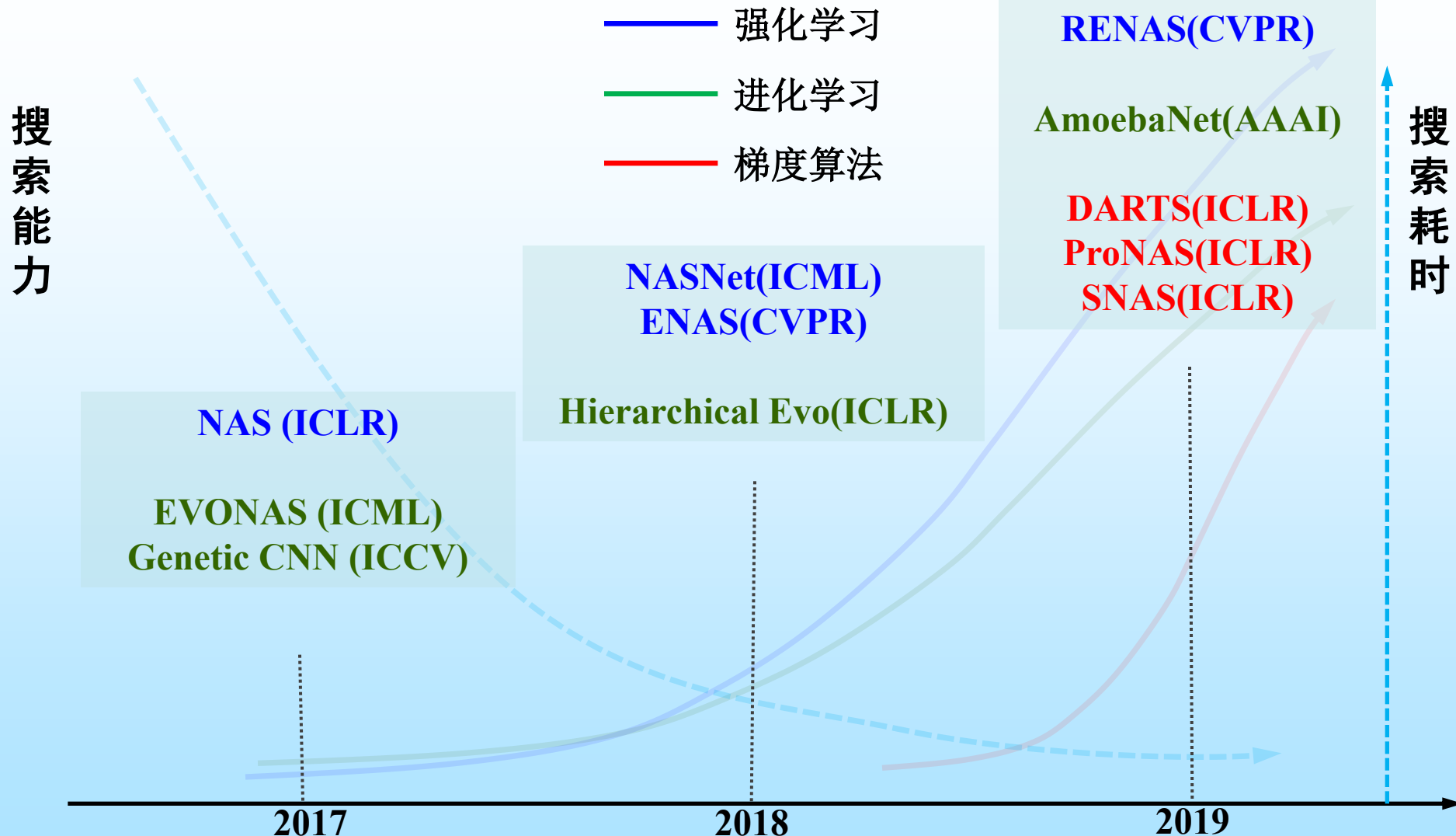


Chen, Y., et al. RENAS: Reinforced Evolutionary Neural Architecture Search. CVPR 2019.

Zhang, X., et al. Single Shot Neural Architecture Search via Direct Sparse Optimization. TNNLS (submitted).

Chang, J., et al. DATA: Differentiable ArchiTecture Approximation. NeruIPS 2019 (submitted).

网络结构搜索

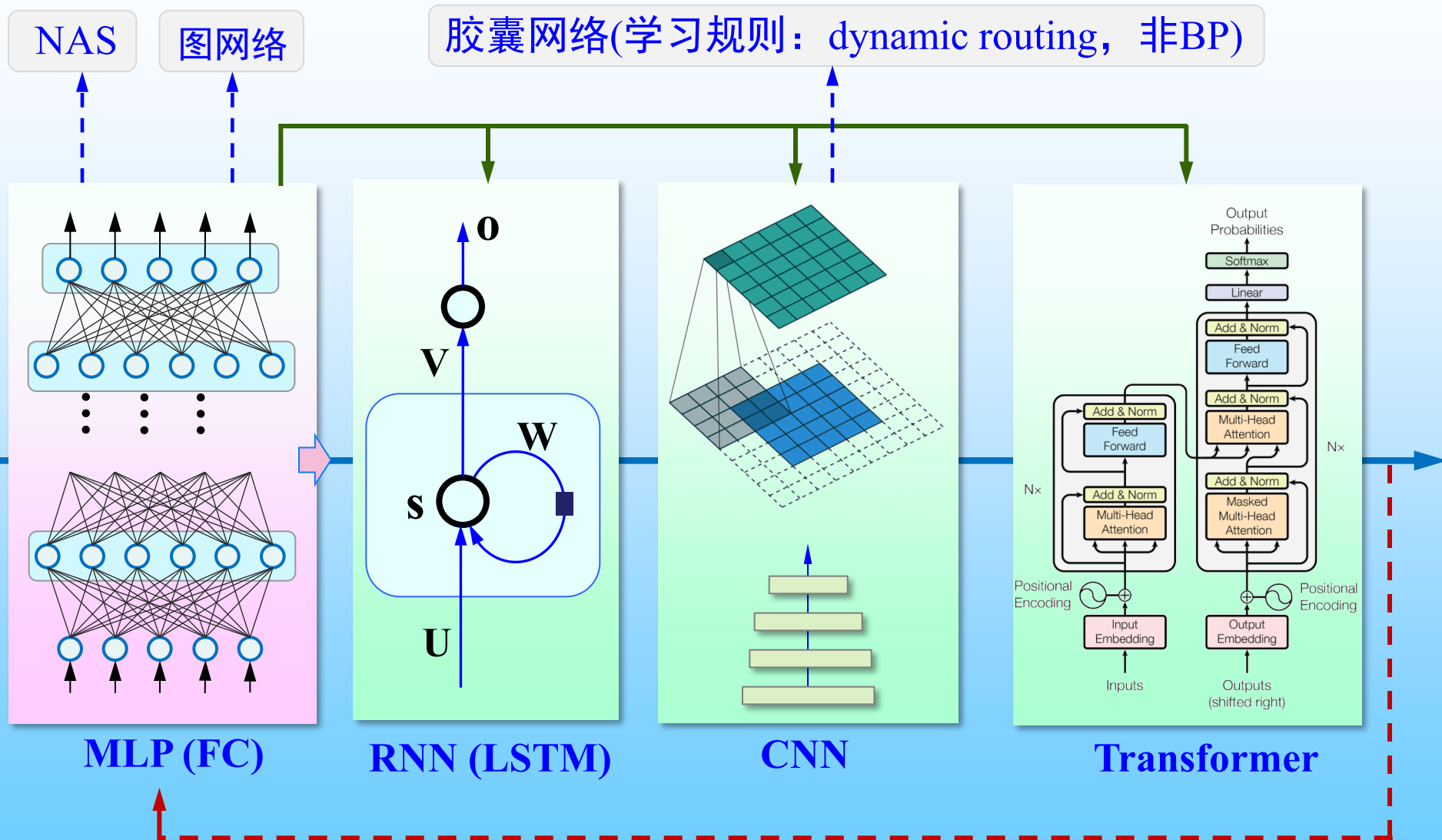


Chen, Y., et al. RENAS: Reinforced Evolutionary Neural Architecture Search. CVPR 2019.

Zhang, X., et al. Single Shot Neural Architecture Search via Direct Sparse Optimization. TNNLS (submitted).

Chang, J., et al. DATA: Differentiable ArchiTecture Approximation. NeruIPS, 2019.

基础构件的演化与发展趋势



致谢

- PPT由向世明老师提供

Thank All of You!
(Questions?)

张燕明

ymzhang@nlpr.ia.ac.cn

people.ucas.ac.cn/~ymzhang

模式分析与学习课题组(PAL)

中科院自动化研究所·多模态人工智能系统实验室